

Trabajo de Fin de Grado

Detección de fraude con GNN frente a métodos tradicionales de *Machine Learning*

Fraud detection with GNNs versus
traditional Machine Learning methods

Samuel Frías Hernández

La Laguna, 7 de julio de 2026

D. **Marcos Colebrook Santamaría**, con N.I.F. 43.787.808-V, profesor Titular de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como tutor.

C E R T I F I C A

Que la presente memoria titulada:

«Detección de fraude con GNN frente a métodos tradicionales de Machine Learning»

ha sido realizada bajo su dirección por D. **Samuel Frías Hernández**, con N.I.F. 79.356.066-V.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos firman la presente en La Laguna a 7 de julio de 2026.

Agradecimientos

Este trabajo ha contado con el soporte del Convenio de Impulso de la Inteligencia Artificial en Tenerife, suscrito entre la ULL y el Cabildo de Tenerife, y de la Cátedra Cajasieta Big Data, Open Data y Blockchain.

En primer lugar, quiero expresar mi más sincero agradecimiento a mi tutor, Marcos Colebrook, por haberme dado la libertad de elegir el tema de este trabajo y por haberme orientado en todo momento durante su desarrollo. Su disposición a animarme a perseguir mis propias ideas, así como su guía constante para encontrar soluciones ante cada dificultad que ha ido surgiendo, han sido determinantes para el resultado final de esta memoria.

Quiero también dar las gracias a Lucciano y a Carol. Hemos compartido incontables jornadas de trabajo y estudio, en ocasiones los cinco días de la semana, a lo largo de varios cuatrimestres. Gracias a ellos he vivido un grado y una experiencia universitaria muy distintos, y sin duda mucho mejores.

A Lu le debo, entre otras cosas, mi acercamiento a la representación estudiantil, una faceta que descubrí gracias a él y que se ha convertido en una auténtica pasión. Asimismo, me recomendó y animó a utilizar muchas de las herramientas que han resultado claves en el desarrollo de este trabajo, y siempre me ha exigido un nivel de rigor que me ha hecho crecer como estudiante.

A Carol le agradezco haberme «obligado», en el mejor sentido, a plantear un trabajo ambicioso, que fuera más allá del desarrollo de una simple herramienta utilitaria, y a buscar un tutor a la altura de ese reto. Su presencia en mi vida me ha hecho mejor persona en muchos aspectos.

Licencia



This work is licensed under CC BY-NC-SA 4.0.

Resumen

La detección de fraude financiero, y en particular el blanqueo de capitales, presenta un desafío analítico que exige evolucionar desde la evaluación transaccional aislada hacia enfoques capaces de auditar estructuras relacionales complejas. En este trabajo se presenta una evaluación comparativa exhaustiva entre algoritmos del estado del arte en *Machine Learning* tradicional y arquitecturas de aprendizaje profundo sobre grafos. Mediante la experimentación empírica sobre el simulador sintético IBM AML y el ecosistema financiero real DGraph-Fin, se demuestra que las GNN logran superar las limitaciones de rendimiento de los enfoques tabulares en entornos reales, validando de forma categórica que el fraude es un fenómeno intrínsecamente topológico. No obstante, los resultados también constatan que en escenarios sintéticos, donde la señal de fraude está fuertemente codificada en atributos aislados, los ensamblajes de árboles de decisión mantienen una alta competitividad operativa. Adicionalmente, la investigación integra módulos de Inteligencia Artificial Explicable a través del uso de valores SHAP y del algoritmo GNNExplainer. Esta integración forense ha permitido dotar de transparencia a las predicciones algorítmicas, así como diagnosticar visualmente fenómenos limitantes como el *oversmoothing* causado por el muestreo de vecindarios masivos en los grafos.

Palabras clave: Detección de fraude, Blanqueo de capitales, *Machine Learning*, Redes Neuronales sobre Grafos (GNN), Inteligencia Artificial Explicable (XAI).

Abstract

Financial fraud detection, and particularly money laundering, presents an analytical challenge that demands evolving from isolated transactional evaluation toward approaches capable of auditing complex relational structures. This work presents a comprehensive comparative evaluation between state-of-the-art traditional Machine Learning algorithms and Graph Neural Network architectures. Through empirical experimentation on the synthetic IBM AML simulator and the real-world DGraph-Fin financial ecosystem, it is demonstrated that GNNs successfully overcome the performance limitations of tabular approaches in real environments, categorically validating that fraud is an intrinsically topological phenomenon. However, the results also confirm that in synthetic scenarios, where the fraud signal is heavily encoded in isolated attributes, decision tree ensembles maintain high operational competitiveness. Additionally, this research integrates Explainable Artificial Intelligence modules through the use of SHAP values and the GNNExplainer algorithm. This forensic integration has provided transparency to algorithmic predictions, as well as the ability to visually diagnose limiting phenomena such as oversmoothing caused by massive neighborhood sampling in graphs.

Keywords: Fraud detection, Money laundering, Machine Learning, Graph Neural Networks (GNN), Explainable Artificial Intelligence (XAI).

Índice general

1. Introducción	1
1.1. Anatomía de un Ataque Real	2
1.2. Objetivos del Trabajo	3
2. Antecedentes	5
2.1. Evolución desde el <i>Machine Learning</i> Tradicional	5
2.2. Adopción de <i>Graph Neural Networks</i>	5
2.3. Desafíos Actuales	6
2.4. Datasets y la Necesidad de Explicabilidad	6
3. Desarrollo	7
3.1. Selección del Conjunto de Datos	7
3.1.1. Limitaciones Algorítmicas de los Modelos	7
3.1.2. Criterios Rigurosos para la Selección	8
3.1.3. Resolución Final	10
3.2. Herramientas y Tecnologías	10
3.2.1. Entorno de Ejecución y Gestión del Proyecto	11
3.2.2. Migración a Infraestructura de Alto Rendimiento	11
3.2.3. Procesamiento de Datos y Análisis	12
3.2.4. Modelado de Aprendizaje Automático y Grafos	12
3.2.5. MLOps y Trazabilidad de Experimentos	13
3.2.6. Calidad del Código y Pruebas Automáticas	13
3.2.7. Disponibilidad del Código y Reproducibilidad	13
3.3. Implementación de los Módulos del Sistema	14
3.3.1. Procesamiento de Datos y Construcción del Grafo	14
3.3.2. Modelos de Aprendizaje Automático Tradicional	16
3.3.3. Arquitecturas de Redes Neuronales sobre Grafos	17
3.3.4. Módulo de Explicabilidad Analítica	19
3.3.5. Orquestación, Trazabilidad y Exportación	20
3.3.6. Interfaz de Línea de Comandos y Configuración	21
3.4. Desarrollo de <i>baselines</i> de <i>Machine Learning</i>	24
3.4.1. Adquisición y Preprocesamiento de Datos	24
3.4.2. Configuración y Entrenamiento de los Modelos	25
4. Resultados	26
4.1. Características del Conjunto de Datos IBM AML	26
4.2. Análisis de Modelos de <i>Machine Learning</i>	27
4.3. Análisis de Resultados del Modelo de GNN	28

4.4. Comparativa entre GNN y <i>Machine Learning</i>	30
4.5. Validación de la Hipótesis en DGraph-Fin	31
4.6. Explicabilidad en Modelos de <i>Gradient Boosting</i>	32
4.7. Explicabilidad en Modelos de GNN	34
5. Conclusiones	36
5.1. Líneas de trabajo futuro	37
6. Summary	39
6.1. Future Work	40
7. Presupuesto	42
7.1. Costes de Personal	42
7.2. Costes de <i>Hardware</i>	42
7.3. Costes de <i>Software</i>	43
7.4. Resumen del Presupuesto	43
Bibliografía	44

Índice de figuras

1.1. Representación topológica de un esquema de <i>smurfing</i>	3
4.1. Valores SHAP de XGBoost	33
4.2. Valores SHAP de LightGBM	33
4.3. Aristas más importantes según la explicación de la GNN	35

Índice de tablas

4.1.	Comparativa de las variantes del conjunto de datos IBM AML	26
4.2.	Resultados de los modelos tradicionales sobre el conjunto IBM AML	28
4.3.	Resultados del modelo GNN con distintos hiperparámetros	29
4.4.	Comparativa de rendimiento sobre IBM AML	30
4.5.	Comparativa de rendimiento sobre DGraph-Fin	32
7.1.	Presupuesto de personal	42
7.2.	Presupuesto de <i>Hardware</i>	43
7.3.	Presupuesto total	43

Capítulo 1

Introducción

La detección automática de fraude financiero es un desafío analítico y computacional crítico en la minería de datos, la ciberseguridad y el aprendizaje automático. El panorama de las transacciones financieras ha incrementado su complejidad de manera exponencial durante la última década, impulsado por la integración económica global, la adopción masiva de pasarelas de pago digitales, y los continuos avances en las tecnologías de la información [1].

Esta hiperconectividad, aunque favorece el comercio legítimo, amplía significativamente la superficie de ataque, propiciando esquemas fraudulentos más sofisticados y difíciles de rastrear mediante heurísticas convencionales. A nivel macroeconómico, se estima que las pérdidas mundiales por fraude con tarjetas de crédito alcanzarán los 403.88 mil millones de dólares durante la próxima década [2].

En un espectro delictivo aún más amplio, la Oficina de las Naciones Unidas contra la Droga y el Delito estima que el blanqueo de capitales, una tipología de fraude que involucra el movimiento de fondos ilícitos para ocultar sus orígenes, representa entre el 2 % y el 5 % del Producto Interior Bruto (PIB) mundial, lo que se traduce en un volumen de lavado que oscila entre los 0.8 y 2.0 billones (10^{12}) de dólares anuales [3].

Frente a esta amenaza sistémica, la industria financiera y la comunidad académica han dependido históricamente de enfoques estadísticos y de aprendizaje automático tradicional (*Machine Learning*), tales como la Regresión Logística, las Máquinas de Vectores de Soporte (SVM), los Árboles de Decisión, y los potentes métodos por conjuntos como *Random Forest* y *Gradient Boosting* (por ejemplo, XGBoost o LightGBM). Estos sistemas han demostrado ser eficaces hasta cierto punto, ofreciendo capacidades predictivas robustas cuando se enfrentan a patrones de fraude aislados o comportamientos atípicos evidentes en una única transacción [4].

Sin embargo, la naturaleza misma del fraude financiero moderno ha evolucionado desde incidentes simples e inconexos hacia redes delictivas altamente organizadas e interconectadas. Los defraudadores profesionales operan mediante redes de mulas de dinero, la reutilización coordinada de dispositivos móviles comprometidos, la creación de identidades sintéticas, y esquemas de estratificación de blanqueo de capitales que involucran cadenas complejas de transferencias diseñadas matemáticamente para ofuscar el origen de los fondos ilícitos [5].

La limitación fundamental de los modelos clásicos de aprendizaje automático reside en su ceguera topológica. Estos algoritmos están diseñados para operar sobre datos tabulares, donde cada fila (una transacción) se trata como un vector de características independiente en un espacio euclidiano [2]. Al analizar un evento de forma aislada, un modelo como XGBoost puede determinar que una transferencia de cincuenta dólares desde una dirección IP residencial no viola ninguna regla de negocio estática ni presenta anomalías estadísticas en su importe [6]. El modelo tabular no captura la red de relaciones subyacentes, como el hecho de que una misma dirección IP haya sido compartida por cincuenta cuentas de usuario distintas en las últimas dos horas para transferir cantidades idénticas a una cuenta centralizadora [5].

Para superar esta limitación, las Redes Neuronales sobre Grafos (*Graph Neural Networks* o GNN) se han consolidado como un enfoque destacado en la literatura científica reciente sobre detección de anomalías [1]. Las GNN poseen la capacidad única e inherente de aprender representaciones vectoriales densas (*embeddings*) a partir de datos estructurados en forma de redes no euclidianas. Modifican el enfoque predictivo al modelar de manera explícita y matemática las interacciones complejas entre nodos (entidades como usuarios, tarjetas de crédito, comercios o dispositivos) y aristas (relaciones dinámicas como transacciones, accesos compartidos o transferencias de capital). A través de un proceso iterativo conocido como paso de mensajes (*message passing*) o convolución de grafos, las GNN permiten que la evaluación del riesgo de una transacción específica esté informada contextualmente por el comportamiento histórico y la topología de todas las entidades vecinas en el grafo, extendiéndose a múltiples «saltos» (*hops*) de distancia [4].

1.1. Anatomía de un Ataque Real

Para comprender la imperiosa necesidad de evolucionar hacia arquitecturas basadas en grafos, resulta fundamental diseccionar la mecánica operativa de los esquemas modernos de blanqueo de capitales. Lejos de constituir anomalías estadísticas evidentes o transacciones aisladas, el lavado de activos se orquesta como un proceso multifase continuo: colocación, estratificación e integración. Es precisamente en la fase de estratificación donde las redes delictivas despliegan topologías transaccionales altamente intrincadas para desvincular los fondos ilícitos de su origen [5].

Dentro de esta fase, una de las tipologías criminales más prevalentes y sofisticadas es el conocido *pitufeo* o *smurfing*. Esta técnica explota deliberadamente la rigidez de los umbrales estadísticos y las reglas heurísticas de los Sistemas de Monitoreo de Transacciones (TMS) convencionales. En lugar de transferir una suma masiva de capital que activaría de forma inmediata las alarmas de lavado de dinero, la organización criminal fragmenta el monto total en miles de depósitos microscópicos.

Estos micro-depósitos son canalizados a través de una densa red de intermediarios, comúnmente denominados «cuentas mula» (*mule accounts*). Estas entidades, que frecuentemente corresponden a identidades sintéticas o a usuarios legítimos comprometidos, actúan como nodos de paso en el ecosistema financiero. Desde la perspectiva de un modelo tabular clásico, cada transferencia individual inter-mula se proyecta como una operación bancaria cotidiana, de muy bajo importe y exenta de riesgo aparente, logrando evadir por completo el escrutinio de los clasificadores lineales.

Sin embargo, la amenaza sistemática del *smurfing* se revela de forma flagrante al observar el flujo del capital desde un paradigma relacional. Tras una serie coordinada de saltos transaccionales (*hops*) diseñados exclusivamente para maximizar la ofuscación del rastro financiero, la red de mulas converge. Los fondos, previamente atomizados, se reensamblan mediante transferencias orquestadas hacia una o varias cuentas centralizadoras, radicadas habitualmente en jurisdicciones extraterritoriales de laxitud regulatoria (paraísos fiscales) o convertidas en criptoactivos no trazables, culminando así la fase de integración.

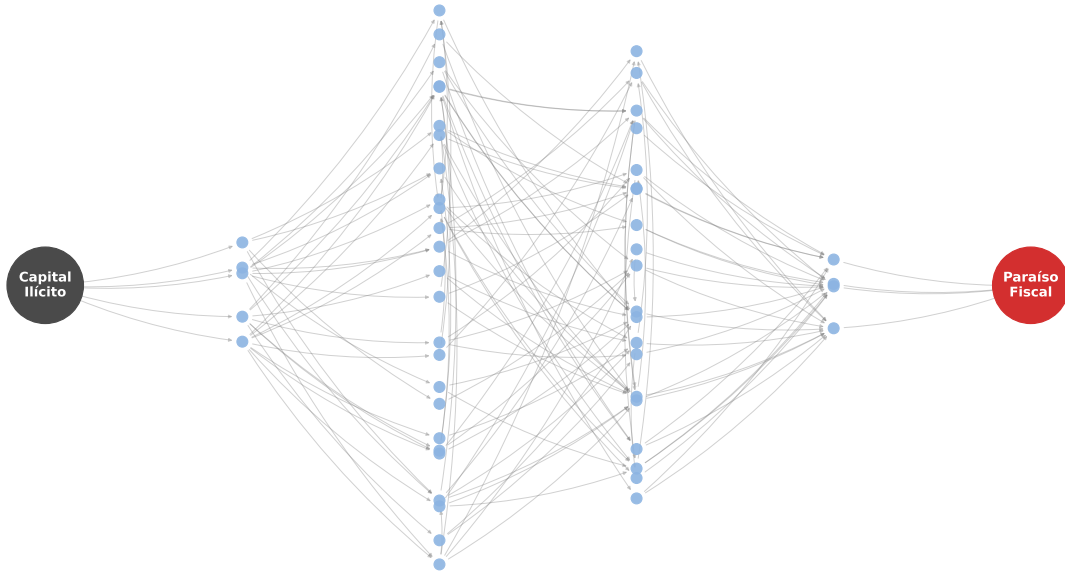


Figura 1.1: Representación topológica de un esquema de *smurfing*

La ceguera ante esta coreografía delictiva es precisamente lo que lastra a los algoritmos de *Machine Learning* tradicional [1]. Un estimador estadístico que evalúa cada fila tabular de forma independiente y ortogonal es matemáticamente incapaz de percibir que cientos de transacciones, espacial y temporalmente dispersas, forman parte de un macro-evento coordinado. Por consiguiente, la neutralización de tipologías topológicas complejas como el *smurfing* exige abandonar el espacio euclidiano y adoptar modelos predictivos capaces de auditar la red subyacente de manera nativa.

1.2. Objetivos del Trabajo

El objetivo general de este trabajo consiste en diseñar, implementar y evaluar empíricamente arquitecturas de Redes Neuronales sobre Grafos para la detección de esquemas complejos de blanqueo de capitales, estableciendo una comparativa de rendimiento exhaustiva frente a modelos tradicionales de aprendizaje automático. El proyecto persigue demostrar cómo la integración sistemática de características topológicas enriquece la capacidad predictiva del sistema, permitiendo superar la inherente ceguera topológica de los enfoques analíticos tabulares convencionales.

Objetivos Específicos

Para materializar el propósito general, el desarrollo se articula en torno a los siguientes objetivos específicos:

1. **Ingesta y modelado estructural de datos financieros:** desarrollar un *pipeline* robusto para la transición de representaciones tabulares planas hacia un modelado de red complejo. Esto comprende el despliegue de infraestructura en contenedores para integrar la base de datos orientada a grafos, modelando las cuentas bancarias como nodos y las transacciones financieras como aristas direccionales temporales.
2. **Ingeniería de características geométricas y relacionales:** extraer el contexto estructural subyacente del conjunto de datos. Este objetivo requiere la ejecución algorítmica de métricas de centralidad, detección de comunidades y la generación de *embeddings* topológicos para aislar comportamientos reticulares propios de la fase de estratificación, tales como cuentas mula o cuellos de botella.
3. **Construcción de modelos tabulares de referencia:** implementar y optimizar modelos heurísticos y de ensamblaje tradicionales. Estos sistemas operarán como línea base empírica para evidenciar las deficiencias predictivas derivadas de tratar las transacciones como vectores de características independientes en un espacio euclidiano.
4. **Diseño y entrenamiento de arquitecturas GNN:** desarrollar modelos de aprendizaje profundo geométrico adaptados a la intrincada topología de los multigrafos dirigidos financieros. El diseño debe estar orientado a mitigar las limitaciones teóricas del paso de mensajes estándar, capturando eficazmente los patrones de ofuscación y el fraccionamiento de capital que habitualmente eluden a los Sistemas de Monitoreo de Transacciones (TMS) estáticos.
5. **Evaluación cuantitativa y contraste de rendimiento:** ejecutar una comparativa analítica rigurosa entre el ecosistema tabular y el paradigma relacional. Se incidirá de manera crítica en la optimización de métricas robustas frente al severo desbalanceo de clases inherente al fraude financiero, cuantificando la mejora predictiva derivada de la inyección de contexto estructural precalculado en la red.

Capítulo 2

Antecedentes

La detección de fraude financiero ha experimentado una evolución significativa en la última década, impulsada por la creciente sofisticación de los ataques y el volumen masivo de transacciones digitales. Históricamente, las instituciones financieras han dependido de sistemas basados en reglas y algoritmos de *Machine Learning* tradicional. Sin embargo, la literatura reciente refleja una transición clara hacia los modelos basados en el aprendizaje profundo de grafos (GNN) para abordar las limitaciones de los enfoques convencionales.

2.1. Evolución desde el *Machine Learning* Tradicional

Los métodos tradicionales de aprendizaje automático supervisado han sido el estándar en la industria durante años debido a su alta precisión tabular [7]. Recientemente, autores como Almalki y Masud han demostrado que el uso de ensamblajes apilados (*stacking ensembles*) combinando XGBoost, LightGBM y CatBoost sigue ofreciendo resultados competitivos en la detección de anomalías crediticias [8].

No obstante, estos modelos tabulares presentan una limitación estructural fundamental: asumen que las transacciones son eventos independientes. Al aplanar los datos para su procesamiento, ignoran la rica información topológica y las relaciones subyacentes entre las entidades involucradas (usuarios, cuentas, dispositivos, etc.), lo cual es crucial dado que los defraudadores modernos operan en redes coordinadas y complejas [1].

2.2. Adopción de *Graph Neural Networks*

Para superar la falta de contexto relacional, la comunidad científica ha volcado sus esfuerzos en las *Graph Neural Networks*. Las GNN han demostrado una capacidad excepcional para capturar patrones relacionales y dinámicas dentro de redes financieras, superando significativamente a los modelos no relacionales en diversas métricas de evaluación [1].

Estudios comparativos directos respaldan esta transición. Por ejemplo, Lu propuso un modelo integral que combina Redes Convolucionales de Grafos (GCN) y Redes de Atención de Grafos (GAT) para la agregación jerárquica de características, demostrando sobre conjuntos de datos públicos (como el IEEE-CIS) que las GNN superan con creces a la Regresión Logística, SVM y *Random Forest* en términos de precisión y *F1-Score* [7]. Este

tipo de evidencias fundamentan la necesidad de evaluar y comparar sistemáticamente ambas familias de algoritmos en entornos controlados.

2.3. Desafíos Actuales

A pesar de su éxito, la aplicación de GNN en la detección de fraude se enfrenta a desafíos intrínsecos del dominio financiero que la literatura actual intenta mitigar:

- **Naturaleza tabular de los datos originales:** gran parte de los datos financieros se recopilan en formato tabular, no como grafos explícitos. Investigaciones recientes abordan este problema mediante módulos de aprendizaje métrico que optimizan la construcción del grafo a partir de datos tabulares, asegurando que se cumplan las suposiciones de homofilia (nodos similares tienden a conectarse) necesarias para que las GNN funcionen correctamente [9].
- **Desbalanceo extremo de clases:** las transacciones fraudulentas representan una fracción minúscula frente a las legítimas. Para evitar que las GNN se sesguen hacia la clase mayoritaria, se están explorando técnicas de muestreo híbrido y aprendizaje de adversarios adaptativo, que generan transacciones sintéticas fraudulentas en regiones densas del espacio de características y eliminan ruido, equilibrando así el entrenamiento [9].
- **Detección en tiempo real y dinamismo:** el fraude ocurre de manera instantánea, por lo que los modelos deben inferir de forma rápida y adaptarse a redes que cambian temporalmente. Sultana *et al.* presentaron el modelo **detectGNN**, el cual incorpora patrones basados en el tiempo y actualizaciones dinámicas del grafo para exponer fraudes multicapa de manera escalable [4]. En la misma línea, enfoques híbridos han combinado GNN con *autoencoders* [10] o con técnicas de detección de anomalías no supervisadas [6] para procesar arquitecturas de transmisión de datos en tiempo real, mejorando la prevención inmediata del fraude con tarjetas de crédito.

2.4. Datasets y la Necesidad de Explicabilidad

El avance en este campo se ha visto tradicionalmente obstaculizado por la confidencialidad de los datos bancarios. Aunque existen conjuntos públicos de referencia, la comunidad demanda escenarios más realistas. Ante esto, investigaciones de IBM han desarrollado simuladores para generar transacciones financieras sintéticas realistas orientadas a modelos de prevención de blanqueo de capitales (*Anti Money Laundering*), proporcionando *benchmarks* estandarizados sin comprometer la privacidad [3].

Finalmente, un aspecto crítico para la adopción real de estos modelos en la industria es la transparencia. Las normativas financieras exigen que las decisiones algorítmicas sean interpretables, lo que choca con la naturaleza de «caja negra» del aprendizaje profundo. Mientras que para los modelos tradicionales y de ensamblaje ya se aplican con éxito técnicas de Inteligencia Artificial Explicable (XAI) como SHAP (*SHapley Additive exPlanations*) o LIME [8], la explicabilidad aplicada específicamente a predicciones sobre grafos y GNN sigue siendo un campo emergente y con un alto potencial de investigación. La capacidad no solo de predecir el fraude, sino de identificar qué subgrafo o conexiones exactas motivaron la alarma, representa la frontera actual en la investigación de este campo.

Capítulo 3

Desarrollo

3.1. Selección del Conjunto de Datos

La selección del conjunto de datos es un paso fundamental para comparar la eficacia de las GNN frente a los métodos tradicionales. No todos los datos tabulares pueden transformarse de manera efectiva en un grafo, y no todos los grafos contienen la semántica necesaria para permitir una interpretación comprensible de sus resultados. El análisis se guiará por requisitos sumamente específicos: la necesidad de identificadores relacionales claros que permitan la ingeniería topológica desde cero, la representación fiel del desbalanceo de clases inherente al fraude, y, de manera crucial, la interpretabilidad semántica de las variables para satisfacer la implementación de módulos de explicabilidad (*explainable AI* o XAI) aplicados a inferencias complejas sobre grafos.

3.1.1. Limitaciones Algorítmicas de los Modelos

Los métodos de aprendizaje automático tradicionales, que han sido el pilar de los sistemas anti-fraude bancarios durante las últimas dos décadas, exigen que la información se estructure en una matriz bidimensional plana. En esta matriz, comúnmente almacenada en formatos como CSV o bases de datos relacionales tradicionales, las filas representan las instancias u observaciones (las transacciones individuales) y las columnas representan las características o variables (los atributos de dicha transacción) [10]. Algoritmos como la Regresión Logística, las Máquinas de Vectores de Soporte (SVM), los Árboles de Decisión, y los Perceptrones Multicapa (*Multi-Layer Perceptrons* o MLP) procesan cada vector fila de manera ortogonal e independiente [7].

La eficacia de estos modelos depende en gran medida de la ingeniería de características (*feature engineering*), una fase de procesamiento manual susceptible a introducir sesgos o errores [6]. Para que un modelo *Random Forest* o XGBoost pueda intuir que una transacción forma parte de un ataque coordinado, el investigador debe crear manualmente variables sintéticas temporales y espaciales antes del entrenamiento [11]. Por ejemplo, a partir de un conjunto de datos en bruto, se deben calcular de forma intensiva columnas agregadas como:

- «numero_transacciones_tarjeta_ultimas_24h»,
- «velocidad_gasto_dispositivo_ultimo_mes» o

- «`distancia_geografica_transacciones_consecutivas`».

Si no se anticipa una tipología de fraude emergente y, por tanto, no se genera la variable precalculada correspondiente, el modelo tradicional será completamente ciego ante esa amenaza, independientemente de la profundidad de sus árboles de decisión [6]. Además, estos modelos estadísticos sufren de manera asimétrica ante el problema del desbalanceo de clases; dado que las transacciones fraudulentas a menudo representan menos del 1 % del total en los entornos bancarios reales, los modelos tienden a converger hacia una clasificación mayoritaria constante (prediciendo siempre «no fraude») requiriendo técnicas de sobremuestreo como SMOTE [12] o ADASYN [13], que pueden introducir ruido en los datos y elevar la tasa de falsos positivos en entornos de producción [9].

Las Redes Neuronales sobre Grafos modelan los datos directamente como un grafo $G = (V, E)$, lo que permite procesar la topología y las interconexiones de las transacciones sin necesidad de aplanarlas en una estructura tabular [7].

La innovación fundamental de las GNN es el paradigma de actualización de representaciones basado en la vecindad. En una arquitectura típica de Red Convolutiva de Grafos (*Graph Convolutional Network* o GCN), la representación vectorial oculta de un nodo en la capa $k + 1$ se computa agregando (mediante funciones invariantes a la permutación como la suma, la media o el máximo) las representaciones ocultas de todos sus nodos vecinos en la capa k , combinadas con su propia representación anterior, seguido de una transformación lineal y una función de activación no lineal [7].

Este proceso permite que la información sobre el comportamiento sospechoso fluya a través de la red de pagos. Si una cuenta bancaria legítima transfiere dinero accidentalmente a una entidad marcada previamente como fraudulenta (una mula de dinero conocida), la GNN propagará esa «señal de riesgo» a través de la arista que las conecta. En las capas subsecuentes, la representación embebida de la cuenta originalmente legítima se ajustará matemáticamente para reflejar su proximidad estructural a un clúster de actividad delictiva, permitiendo una clasificación preventiva antes de que el fraude se materialice por completo [1].

Para ejecutar una comparativa empírica rigurosa entre estos dos mundos analíticos, el conjunto de datos elegido debe, paradójicamente, satisfacer ambos paradigmas. Debe poseer características tabulares continuas y categóricas ricas para entrenar las líneas base de XGBoost y SVM, y simultáneamente debe contener el rastro topológico relacional explícito para permitir la inferencia de las matrices de adyacencia necesarias para las GNN.

3.1.2. Criterios Rigurosos para la Selección

Tras una revisión exhaustiva de la literatura, se establecen cuatro criterios innegociables que el *dataset* óptimo debe satisfacer simultáneamente para un análisis que incluya módulos de explicabilidad algorítmica.

1. Existencia de Identificadores Relacionales Unívocos

El *dataset* no requiere presentarse en un formato de grafo precompilado (como archivos de listas de aristas de NetworkX o grafos serializados de PyTorch Geometric), pero es absolutamente imperativo que los datos tabulares sin procesar incluyan identificadores

relacionales persistentes. Estos identificadores son las «claves primarias» y «claves ajenas» conceptuales que conectan entidades dispares a lo largo del tiempo. Un *dataset* tabular que simplemente liste «Transacción_ID, Cantidad, Fecha, Etiqueta_Fraude» es topológicamente estéril e inservible para GNN. El *dataset* debe contener obligatoriamente campos como «Cuenta_Origen», «Cuenta_Destino», «ID_Dispositivo_Cliente», «MAC_Address», «Correo_Electrónico» o «Hash_Tarjeta» [5].

Estos identificadores son la materia prima heurística que permitirá, en fases posteriores de la investigación, establecer las conexiones matemáticas que conforman las aristas del grafo. Si el cliente *A* transfiere al cliente *B*, y el cliente *B* transfiere al cliente *C*, los identificadores unívocos permitirán al algoritmo ensamblar una cadena dirigida $A \rightarrow B \rightarrow C$, revelando una potencial tipología de interacción de lavado de dinero [14].

2. Transparencia Semántica e Interpretabilidad de Atributos

Este es el criterio más restrictivo y el que descalifica a la mayoría de los conjuntos de datos del mundo real publicados por instituciones bancarias. Un objetivo fundamental de la investigación contemporánea es explorar la interpretabilidad mediante técnicas de explicabilidad (XAI) aplicadas a GNN. Cuando un banco comercial dona un conjunto de datos a la comunidad científica, lo hace tras un agresivo proceso de ofuscación de datos y Análisis de Componentes Principales (PCA) para cumplir estipulaciones de privacidad como el GDPR o normativas de la SEC. El resultado es un *dataset* donde las columnas se etiquetan como $V_1, V_2, \dots, V_n$, y los valores son proyecciones numéricas continuas sin significado humano [11].

Para un algoritmo XGBoost orientado exclusivamente a maximizar el Área Bajo la Curva ROC (*Area Under the Receiver Operating Characteristic* o AUC), esta ofuscación es irrelevante, ya que el álgebra lineal subyacente sigue funcionando. Sin embargo, para un proyecto de investigación que persigue la explicabilidad, es catastrófico. Si una red neuronal sobre grafos entrenada y sometida a un algoritmo como GNNExplainer determina que un nodo fue clasificado como fraudulento porque el nodo vecino tenía un valor anómalo en la característica V_{42} , el hallazgo es semánticamente vacío y carece de aplicabilidad en el mundo real. Por consiguiente, el *dataset* ideal debe preservar la integridad semántica de sus columnas: las cantidades deben ser monetarios, las fechas deben ser marcos temporales decodificables, y las categorías de comerciantes deben ser comprensibles (ej., «Joyería», «Supermercado», «Criptomonedas») [8]. Solo así el módulo de explicabilidad podrá concluir inferencias analíticas útiles, del tipo: «El modelo predijo un 99 % de riesgo de fraude debido a una estructura en estrella donde un mismo dispositivo móvil intentó compras simultáneas de alto valor en joyerías internacionales» [5].

3. Fiel Representación del Desbalanceo de Clases

La detección de anomalías es fundamentalmente el estudio del desbalanceo extremo. En los sistemas de pagos globales, el fraude es afortunadamente un evento raro, típicamente representando entre el 0.1 % y el 5 % del volumen transaccional total [11]. Un conjunto de datos artificialmente equilibrado (50 % fraude, 50 % legítimo) induciría a un sesgo metodológico masivo, resultando en un modelo que sufriría un colapso de precisión (*precision collapse*) en un entorno de producción, generando miles de falsos positivos y paralizando la experiencia de usuario [2].

El *dataset* seleccionado debe reflejar esta asimetría orgánica. Esta disparidad obliga al marco de evaluación de la investigación a abandonar métricas engañosas como la exactitud simple (*accuracy*) y adoptar métricas robustas frente al desbalanceo, tales como el Área Bajo la Curva *Precision-Recall* (PR-AUC), la Puntuación F1 (*F1-Score*), y matrices de confusión detalladas. El desafío para las GNN será demostrar su superioridad sobre *Random Forest* precisamente en estas métricas críticas, localizando la escasez de anomalías en un inmenso mar de nodos lícitos.

4. Accesibilidad, Escala y Formato Abierto

Finalmente, para la ejecución práctica de un Trabajo de Fin de Grado (TFG) o investigación independiente, el conjunto de datos debe ser intrínsecamente público, libre de licencias comerciales restrictivas (como NDA bancarios) y poseer un volumen estadísticamente significativo. Conjuntos de datos con solo mil registros, como *Fraud Guard Synthetic 2025* [15], son excelentes para tutoriales de pregrado y depuración de código, pero carecen de la escala necesaria para que el entrenamiento profundo de una arquitectura GAT revele ventajas sobre modelos estadísticos superficiales. Se requiere un *dataset* del orden de cientos de miles o millones de registros tabulares para que la red neuronal tenga suficientes ejemplos para optimizar sus hiperparámetros y converger adecuadamente sin caer en el sobreajuste (*overfitting*) masivo [11].

3.1.3. Resolución Final

Para cumplir con los requisitos de esta investigación, se ha seleccionado el *IBM Transactions for Anti Money Laundering (AML) Dataset*, publicado en NeurIPS 2023 [3], como el conjunto de datos de referencia.

El *dataset* IBM AML resuelve las limitaciones descritas anteriormente, proporcionando una matriz tabular extensa con un desbalanceo natural de clases y sin variables ofuscadas [16]. Incluye los identificadores relacionales explícitos (banco, origen, receptor, divisa) necesarios para construir el grafo heterogéneo $G(V, E, R)$ durante la fase de preprocesamiento. En esta formalización geométrica, V representa el conjunto de vértices o entidades involucradas (como las cuentas bancarias), E denota las aristas que materializan las transacciones, y R define los múltiples tipos de relaciones o interacciones direccionales que dotan de heterogeneidad a la red. Esta estructura permite que el módulo de explicabilidad identifique de forma comprensible redes complejas de blanqueo de capitales, una tarea estructuralmente inalcanzable para modelos tabulares basados en árboles como XGBoost.

3.2. Herramientas y Tecnologías

El desarrollo de esta investigación se apoya en un ecosistema tecnológico moderno, diseñado para garantizar la eficiencia computacional, la reproducibilidad de los experimentos y la escalabilidad del análisis sobre grandes volúmenes de datos financieros. A continuación, se detallan las principales herramientas y bibliotecas empleadas, estructuradas según su rol en la arquitectura del proyecto.

3.2.1. Entorno de Ejecución y Gestión del Proyecto

Para asegurar la consistencia del entorno de desarrollo independientemente de la máquina de ejecución, el proyecto se ha contenerizado utilizando la tecnología Docker a través de DevContainers [17]. Este enfoque define un entorno aislado basado en Python 3.11, donde todas las dependencias a nivel de sistema y paquetes de *software* se instalan de forma automatizada, eliminando el problema clásico de incompatibilidad de versiones.

Adicionalmente, el entorno local se orquesta mediante Docker Compose para desplegar un arquitectura multicontenedor. Esto incluye un servicio dedicado de Neo4j, configurado con restricciones de memoria específicas y dotado de los *plugins Graph Data Science* [18] y APOC, lo cual resulta indispensable para persistir la topología de la red de transacciones.

La gestión de dependencias y el empaquetado del código fuente se ha estandarizado mediante un archivo `pyproject.toml`, utilizando `setuptools` como sistema de construcción, lo que facilita la instalación del proyecto como un paquete modular.

3.2.2. Migración a Infraestructura de Alto Rendimiento

El principal obstáculo técnico para la consecución de los objetivos de esta investigación radicaba en las dimensiones y la exigencia computacional inherente al conjunto de datos seleccionado. El entrenamiento de arquitecturas de aprendizaje profundo sobre grafos (GNN), que requieren cargar matrices de adyacencia dispersas y vectores de características de cientos de miles de nodos simultáneamente en memoria, supone una barrera técnica insalvable para equipos de desarrollo a nivel local de recursos limitados.

Para solventar el problema del coste computacional y asegurar la viabilidad de los experimentos, se estableció una colaboración con la Cátedra Big Data, Open Data y Blockchain (BOB) de la Universidad de La Laguna [19]. Tras exponer las necesidades del proyecto, la Cátedra facilitó el acceso a su infraestructura de investigación, garantizando el uso de máquinas equipadas con aceleración NVIDIA DGX Spark, un *hardware* de supercomputación diseñado específicamente para cargas de trabajo de inteligencia artificial masivas.

Adaptación del Entorno y Arquitectura de Ejecución

La utilización de esta infraestructura compartida requirió una adaptación del entorno de ejecución original del proyecto. Mientras que el desarrollo local se gestionaba mediante un DevContainer estándar, el despliegue en la máquina DGX exigió integrar el repositorio público de este trabajo dentro de un entorno Docker estructurado (el *boilerplate* proporcionado por la Cátedra BOB).

Esta adaptación introduce una separación arquitectónica estricta:

- **Contenedor Base de Alto Rendimiento:** se utiliza una imagen Docker pre-configurada (`nvidia/cuda:12.9.0-devel-ubuntu24.04`) que inyecta directamente los controladores de NVIDIA y el Container Toolkit, habilitando el acceso sin restricciones a las GPU Blackwell disponibles.
- **Encapsulamiento del Código:** el repositorio de código del TFG se anida íntegramente dentro de un directorio de montaje, el cual se sincroniza como `/workspace` en el contenedor. Esto garantiza que la lógica del modelo y los *scripts* de entrenamiento permanezcan agnósticos a la máquina subyacente, ejecutándose en un entorno virtual

aislado con sus dependencias instaladas dinámicamente mediante el *entrypoint* del contenedor.

Monitorización en Directo y Exportación Automatizada

Dado que las políticas de seguridad de la infraestructura DGX Spark limitan el acceso interactivo directo, fue imperativo diseñar un flujo de trabajo que permitiese la monitorización remota y la recuperación asíncrona de los modelos entrenados.

Para abordar este reto operativo, se implementaron dos soluciones integradas:

- **Servidor de Logs en Tiempo Real:** en paralelo al contenedor de entrenamiento, el entorno despliega mediante `docker-compose` un servidor web ligero basado en FastAPI [20]. Este servicio expone un *endpoint* protegido por *token* a través del puerto 8000, permitiendo la monitorización en vivo del flujo de salida estándar directamente desde un navegador web, sin necesidad de acceso SSH.
- **Sincronización de Artefactos en la Nube:** para la recuperación de los resultados, se desarrolló el módulo `DataSyncManager`, acoplado al sistema de seguimiento de MLflow. Al finalizar la canalización (*pipeline*) de entrenamiento y evaluación, el sistema empaqueta automáticamente la base de datos de experimentos y modelos serializados, y los sube a un repositorio privado en Hugging Face Hub utilizando la API de la plataforma.

Este diseño arquitectónico ha resultado clave, ya que reduce la intervención del personal de la Cátedra a un simple comando de arranque, automatizando por completo el registro, el empaquetado y la exportación de resultados, y dotando a la investigación de un ciclo de experimentación ágil, escalable y totalmente desatendido.

3.2.3. Procesamiento de Datos y Análisis

El procesamiento de conjuntos de datos financieros requiere herramientas capaces de manejar millones de registros con alta eficiencia de memoria. Por ello, se ha adoptado Polars [21] como motor principal para la manipulación de datos tabulares. A diferencia de bibliotecas tradicionales, Polars está implementado en Rust y utiliza procesamiento multihilo y evaluación perezosa (*lazy evaluation*), lo que acelera significativamente las fases de limpieza y filtrado de los datos transaccionales.

Para la compatibilidad con los algoritmos de *Machine Learning* y la división de los conjuntos de datos (entrenamiento, validación y prueba), se ha empleado la biblioteca Scikit-learn [22]. Específicamente, sus módulos de preprocesamiento han sido fundamentales para la codificación de variables categóricas (*Label Encoding*) conservando la consistencia algorítmica.

3.2.4. Modelado de Aprendizaje Automático y Grafos

El núcleo analítico de la investigación compara modelos predictivos clásicos frente a arquitecturas de aprendizaje profundo sobre grafos. Para las líneas base (*baselines*) de *Machine Learning* tradicional, se han integrado las implementaciones oficiales de los métodos de potenciación del gradiente (*Gradient Boosting*): XGBoost [23] y LightGBM [24]. Estos algoritmos representan el estado del arte actual en la industria para datos tabulares.

Adicionalmente, se incluyen clasificadores estándar como *Random Forest* y Máquinas de Vectores de Soporte (SVM) provenientes de Scikit-learn.

Para el desarrollo de las Redes Neuronales sobre Grafos, el proyecto integra PyTorch [25] como marco principal de aprendizaje profundo, apoyado en la extensión PyTorch Geometric (PyG) [26]. PyG proporciona las primitivas necesarias para implementar arquitecturas de convolución de grafos (GCN) y redes de atención de grafos (GAT) de forma altamente optimizada.

Para abordar ciertas limitaciones de rendimiento en los modelos y mejorar las métricas de evaluación, el ecosistema se ha ampliado con la integración de Neo4j y la librería oficial `graphdatascience`. El uso de estas tecnologías, combinado con herramientas de investigación automatizada, facilita la exploración topológica y la extracción ágil de características complejas directamente desde la base de datos de grafos.

De cara al análisis de interpretabilidad, la arquitectura contempla el uso de herramientas de Inteligencia Artificial Explicable (XAI) como SHAP [27], que permitirá auditar las decisiones tomadas tanto por los modelos de ensamble como por las GNN.

3.2.5. MLOps y Trazabilidad de Experimentos

Dada la gran cantidad de hiperparámetros y modelos evaluados, es imperativo contar con un sistema robusto de seguimiento de experimentos (MLOps). Para ello, se ha integrado MLflow [28]. Este marco registra automáticamente las configuraciones de cada ejecución, las métricas de evaluación de rendimiento (como el *F1-Score* y el Área Bajo la Curva PR) y serializa los modelos entrenados en una base de datos SQLite local.

Para la persistencia a largo plazo y la sincronización en la nube, se han utilizado las bibliotecas Hugging Face Hub [29] y Kagglehub [30]. Estas herramientas permiten descargar los conjuntos de datos en bruto de forma programática y subir los artefactos generados (como los registros comprimidos de MLflow) a repositorios remotos, garantizando la preservación total de los resultados de la investigación.

3.2.6. Calidad del Código y Pruebas Automáticas

Finalmente, para mantener el rigor y la fiabilidad del software desarrollado, se ha implementado un flujo de Aseguramiento de la Calidad (QA). Se utiliza Ruff [31] como *linter* y formateador extremadamente rápido, y el *framework* Pytest [32] para la ejecución de pruebas unitarias sobre los módulos críticos, como los gestores de datos y registros.

3.2.7. Disponibilidad del Código y Reproducibilidad

Para garantizar la total transparencia metodológica y permitir la reproducibilidad de los experimentos documentados en esta memoria, la totalidad del código fuente ha sido liberada bajo licencia de código abierto. El ecosistema completo se encuentra alojado y versionado en un repositorio público de GitHub, accesible a través del siguiente enlace: [SamuelFriHer/TFG-Fraud-Detection-GNN](https://github.com/SamuelFriHer/TFG-Fraud-Detection-GNN).

3.3. Implementación de los Módulos del Sistema

3.3.1. Procesamiento de Datos y Construcción del Grafo

El primer paso fundamental en la arquitectura del sistema es la ingesta, limpieza y transformación de los datos crudos hacia una estructura que soporte tanto el modelado tabular clásico como el aprendizaje profundo sobre grafos. Este proceso se ha dividido en tres fases principales: transformación tabular, extracción de características topológicas mediante Neo4j y ensamblaje final de la estructura relacional.

Limpieza y Transformación Tabular

Para manejar de forma eficiente el volumen masivo de registros que presenta el conjunto de datos IBM AML, se ha implementado un módulo de preprocesamiento, denominado **DataPreprocessor**, utilizando la librería Polars. Esta elección responde a su naturaleza de procesamiento multihilo y a la evaluación perezosa (*lazy evaluation*), lo que minimiza drásticamente el consumo de memoria y el tiempo de cómputo durante las operaciones intensivas de proyección y filtrado.

En primer lugar, se identifican las entidades de forma unívoca. Dado que en los datos originales las cuentas bancarias pueden solaparse entre distintas entidades financieras, se genera un identificador sintético concatenando el banco y la cuenta (por ejemplo, combinando las columnas **Bank ID** y **Account Number**). Posteriormente, las variables categóricas, como el tipo de divisa o el formato de pago, son codificadas mediante *Label Encoding*, preservando un diccionario de mapeo interno (**encoders**) para garantizar la consistencia analítica durante las inferencias y en el momento de aplicar la explicabilidad posterior.

Ingesta en Neo4j y Extracción Topológica

Una vez que el conjunto tabular está estandarizado, el componente **Neo4jLoader** asume la responsabilidad de volcar la información al servidor de base de datos de grafos. Para prevenir saturaciones de memoria (errores *Out of Memory* o OOM) inherentes a las operaciones masivas de creación de aristas, la población de la base de datos se orquesta mediante cargas iterativas por lotes (*batching*).

A nivel de lenguaje de consulta, se emplean sentencias parametrizadas de Cypher combinadas con la cláusula **UNWIND**. Esta técnica permite transcribir decenas de miles de transacciones aisladas en una única y eficiente transacción de red, agilizando drásticamente la ingesta. El siguiente fragmento de código ilustra esta consulta optimizada:

```

1 query = """
2 UNWIND $batch AS tx
3 MATCH (src:Account {id: tx.from_acc})
4 MATCH (dst:Account {id: tx.to_acc})
5 CREATE (src)-[:TRANSACTIONED {amount: tx.amount, currency: tx.currency}]->(dst)
6 """

```

Listado 3.1: Fragmento de la clase **Neo4jLoader**

El entorno de despliegue, orquestado con Docker, integra adicionalmente los complementos APOC y *Graph Data Science* (GDS). Estas librerías, gestionadas a través del módulo `Neo4jFeatureExtractor`, facilitan el cómputo de métricas topológicas complejas directamente sobre el módulo de Neo4j, las cuales son posteriormente acopladas a la matriz de características de los nodos.

Generación de la Estructura Final

La integración de ambas fuentes recae sobre la clase `AMLGraphBuilder`. La responsabilidad primordial de este módulo es mapear los identificadores relacionales alfanuméricos hacia índices numéricos continuos basados en cero, un requisito matemático indispensable para la representación en forma de matrices dispersas que demanda el entorno de PyTorch.

Para materializar el conjunto, se ensambla el objeto `Data` nativo de Pytorch Geometric (PyG). Este objeto consolida las matrices de adyacencia direccionales (`edge_index`), la matriz de atributos de los nodos previamente enriquecida (`x`), los atributos de las aristas (`edge_attr`) y las etiquetas correspondientes a fraude (`y`).

Resulta imperativo destacar que, para simular de forma fidedigna un entorno financiero de producción y evitar la filtración temporal de información al modelo (*data leakage*), el particionado de los subconjuntos de entrenamiento, validación y prueba se ejecuta respetando estrictamente el orden cronológico de las transacciones mediante la generación condicional de máscaras booleanas.

```

1  def _create_graph_data(
2      self,
3      acc_df: pl.DataFrame,
4      tx_df: pl.DataFrame,
5      test_size: float,
6      neo_df: pl.DataFrame | None,
7  ) -> Data:
8      """Instancia el objeto Data de PyTorch Geometric."""
9      n_edges: int = len(tx_df)
10     train_cutoff: int = int(n_edges * (1.0 - test_size))
11
12     # 1. Extracción de características (nodos y aristas)
13     x: torch.Tensor = self.node_extractor.compute_features(
14         acc_df, tx_df, train_cutoff, neo_df
15     )
16     edge_attr: torch.Tensor = self.edge_extractor.extract_features(tx_df)
17
18     # 2. Cómputo topológico y máscaras cronológicas
19     masks = self._compute_edge_index_and_masks(tx_df, test_size)
20     edge_idx, tr_mask, val_mask, te_mask = masks
21     y: torch.Tensor = torch.tensor(
22         tx_df["Is Laundering"].to_numpy(), dtype=torch.long
23     )
24
25     # 3. Empaquetado final
26     return Data(
27         x=x, edge_index=edge_idx, edge_attr=edge_attr, y=y,
28         train_mask=tr_mask, val_mask=val_mask, test_mask=te_mask,
29     )

```

Listado 3.2: Ensamblaje del grafo en la clase `AMLGraphBuilder`

3.3.2. Modelos de Aprendizaje Automático Tradicional

Para establecer una línea base analítica empírica y rigurosa frente a las Redes Neuronales sobre Grafos, el ecosistema de experimentación incorpora una *suite* completa de modelos predictivos tabulares. La arquitectura de este módulo se fundamenta en principios de diseño orientado a objetos, donde todos los clasificadores tradicionales implementan una interfaz común (`ITraditionalModel`), garantizando su acoplamiento transparente con la canalización de entrenamiento y evaluación.

Ensamblajes Basados en Árboles

Los algoritmos de potenciación del gradiente (*Gradient Boosting*) constituyen el estado del arte en la clasificación de datos tabulares con fuerte desbalanceo. Por consiguiente, se han integrado implementaciones nativas de XGBoost y LightGBM, acompañadas de *Random Forest* como algoritmo estándar de ensamblaje (*bagging*).

Para lidiar con el masivo volumen de transacciones financieras sin incurrir en cuellos de botella computacionales, las clases contenedoras se han diseñado con conciencia de la aceleración por *hardware*. Por ejemplo, la clase `XGBoostModel` sondea dinámicamente el entorno mediante el componente `GpuAvailabilityChecker`; si detecta la disponibilidad de un entorno NVIDIA CUDA, delega el cómputo intensivo del entrenamiento a la tarjeta gráfica de forma automática, reduciendo drásticamente los tiempos de iteración.

Máquinas de Vectores de Soporte (SVM)

La complejidad temporal del algoritmo clásico subyacente en una Máquina de Vectores de Soporte escala de forma cuadrática en función del número de muestras de entrenamiento ($O(n^2)$), lo que convierte su aplicación pura en algo computacionalmente prohibitivo para conjuntos de datos financieros masivos.

Para solventar esta limitación algorítmica y permitir su inclusión en la comparativa, la clase `SVMModel` implementa un patrón de diseño dual apoyado en la biblioteca `cuML` del ecosistema RAPIDS [33]. Si el entorno de ejecución DGX o local cuenta con soporte GPU, el modelo instancia una SVC acelerada por *hardware* que paraleliza el descenso de gradiente. En arquitecturas no compatibles, el sistema se degrada de forma segura hacia la implementación en CPU de `scikit-learn`. El siguiente fragmento de la clase ilustra esta heurística de inicialización:

```

1  if GpuAvailabilityChecker().is_cuml_available():
2      self.classifier, self._backend = self._build_cuml_classifier(defaults)
3  else:
4      self.classifier, self._backend = self._build_sklearn_classifier(defaults)

```

Listado 3.3: Fragmento de la clase `SVMModel`

Canalización del Entrenamiento Tabular

La orquestación del ciclo de vida de estos algoritmos clásicos está centralizada en la clase `TraditionalPipeline`. Esta canalización asegura la reproducibilidad metodológica mediante la ejecución secuencial estricta de las fases de preprocesamiento, particionado, ajuste de pesos y evaluación.

El flujo completo es dirigido mediante archivos de configuración declarativos (TOML), permitiendo la experimentación concurrente y escalable. Entre los mecanismos de ingeniería de *software* que conforman esta canalización destacan:

- **Preprocesamiento Unificado y Eficiente:** mediante el motor de Polars, la ingesta, limpieza y codificación de variables categóricas (*Label Encoding*) se ejecuta de forma centralizada una única vez. El conjunto particionado en tensores de entrenamiento, validación y prueba se inyecta posteriormente a todos los modelos configurados, evitando la redundancia computacional en memoria.
- **Integración Transparente de MLOps:** cada ciclo de entrenamiento se envuelve en un contexto de la plataforma MLflow. El rastreador de experimentos (*ExperimentTracker*) registra sistemáticamente los hiperparámetros declarados, serializa el clasificador binario resultante, y captura métricas críticas frente al desbalanceo, tales como el Área Bajo la Curva PR o la puntuación *F1-Score*.
- **Explicabilidad Algorítmica (XAI):** para los algoritmos basados en árboles de decisión (como XGBoost o LightGBM), la canalización activa de manera condicional el módulo *ShapExplainer*. Tras la inferencia sobre el conjunto de prueba, este componente computa los valores SHAP (*SHapley Additive exPlanations*) de las predicciones, exportando artefactos visuales globales y locales. Esto proporciona la interpretabilidad necesaria para auditar qué atributos tabulares precipitaron exactamente la clasificación de una transacción como fraudulenta.

3.3.3. Arquitecturas de Redes Neuronales sobre Grafos

El desarrollo del componente de detección de fraude basado en grafos fue un proceso iterativo, en el cual se probaron y evaluaron distintas arquitecturas de Redes Neuronales sobre Grafos. Inicialmente, se implementó el modelo GraphSAGE [34], conocido por su escalabilidad computacional en grandes volúmenes de datos. Sin embargo, fue descartado debido a su incapacidad inherente para incorporar características en las aristas (atributos clave como el importe de la transacción, el tipo de moneda o las marcas de tiempo) durante la fase de propagación del mensaje.

Posteriormente, se adoptó la arquitectura GATv2 (*Graph Attention Network v2*) [35], la cual introdujo un mecanismo de atención que permitía a cada nodo ponderar la importancia de sus vecinos de forma dinámica y, sobre todo, integrar las características de las aristas en el cálculo. Aunque los resultados mejoraron notablemente, el modelo seguía presentando limitaciones al procesar multigrafos, es decir, escenarios reales donde existen múltiples transacciones repetidas entre los mismos pares de cuentas.

Para resolver de manera definitiva este desafío, la arquitectura final seleccionada fue MEGA-PNA (*Multi-Edge Graph Aggregation - Principal Neighbourhood Aggregation*) [36]. Este modelo del estado del arte está diseñado específicamente para abordar la agregación de múltiples aristas simultáneas.

Diseño Topológico y Convolución

A nivel topológico, la red procesa primero los *embeddings* iniciales tanto de los nodos (clientes) como de las aristas (transacciones). Cada capa oculta utiliza la operación de convolución definida por PNA, empleando de forma paralela un abanico de agregadores y

escaladores estadísticos para capturar con alta fidelidad las propiedades y varianzas del comportamiento transaccional.

A continuación, se presenta un fragmento del código desarrollado en la clase **MEGAPNAEncoder**, donde se instancian las capas convolucionales:

```

1  for _ in range(config.num_layers):
2      self.convs.append(
3          PNAConv(
4              in_channels=current_in_channels,
5              out_channels=config.hidden_channels,
6              aggregators=["mean", "min", "max", "std"],
7              scalers=["identity", "amplification", "attenuation"],
8              deg=deg,
9              edge_dim=current_edge_dim,
10         )
11     )

```

Listado 3.4: Fragmento de la clase **MEGAPNAEncoder**

En este bloque, se aprecia la inicialización paramétrica mediante la combinación de cuatro operaciones estadísticas diferentes (**mean**, **min**, **max**, **std**) y escaladores por grado, lo que garantiza que las redes extraigan patrones anómalos ocultos en vecindarios transaccionales densos.

Carga de *Minibatches* Espaciales

Dado que las redes de operaciones financieras comprenden millones de entidades y conexiones, resulta técnica y físicamente imposible mantener la estructura completa del grafo en la memoria gráfica (VRAM) de la GPU durante el proceso de entrenamiento. Para solventar este cuello de botella, el sistema implementa un particionado computacional basado en *minibatches* espaciales.

Haciendo uso de la clase **LinkNeighborLoader** de la librería PyTorch Geometric, el grafo se muestrea de manera dinámica. Para cada arista que va a ser analizada en una iteración, se seleccionan aleatoriamente sus nodos vecinos limitando la profundidad de la búsqueda. Esto genera subgrafos computacionales e independientes que retienen estrictamente la conectividad local del contexto original sin incurrir en sobrecargas de memoria.

Entrenamiento y Validación Profunda

El diseño del flujo de entrenamiento requiere una especial rigurosidad topológica al tratar con series cronológicas. Durante la propagación hacia delante y la evaluación de gradientes, es imperativo establecer fronteras temporales estables que prevengan la fuga de datos (*data leakage*). Este paradigma asegura que el modelo, para predecir la legitimidad de una transacción acontecida en un momento T , únicamente tenga acceso estructural a la topología y atributos de momentos anteriores ($T - 1$, $T - 2$, etc.), simulando fielmente un entorno restrictivo de producción.

Finalmente, la fase de validación profunda se centra en mitigar el fuerte desbalanceo de clases característico de los eventos fraudulentos. En lugar de basarse en métricas ingenuas de exactitud (*accuracy*), se integró una rutina anítica que optimiza de manera autónoma

un umbral de decisión dinámico con el objetivo de maximizar la métrica *F1-Score*. De esta forma, el sistema calibra su propia frontera de decisión, garantizando siempre el mejor equilibrio estadístico de precisión y sensibilidad (*recall*).

3.3.4. Módulo de Explicabilidad Analítica

La naturaleza crítica de la detección de fraude financiero, regida por estrictas normativas de cumplimiento, exige que las predicciones algorítmicas no operen como meras «cajas negras». Para satisfacer este requisito fundamental, el ecosistema de evaluación se ha dotado de un módulo de Inteligencia Artificial Explicable (XAI). Este componente aborda la interpretabilidad desde dos frentes metodológicamente diferenciados: la auditoría estadística de los algoritmos tabulares y la disección topológica de las Redes Neuronales sobre Grafos.

Auditoría de Modelos Tabulares

Para los modelos clásicos, especialmente aquellos basados en el ensamblaje de árboles de decisión (XGBoost, LightGBM y *Random Forest*), la explicabilidad se vertebra en torno a los valores SHAP (*SHapley Additive exPlanations*). Esta técnica, fundamentada en la teoría de juegos cooperativos, permite descomponer la inferencia final y asignar a cada característica tabular su contribución exacta (positiva o negativa) hacia la predicción de fraude.

La implementación en la clase `ShapExplainer` prioriza el rendimiento computacional, intentando instanciar inicialmente un `TreeExplainer`, el cual está matemáticamente optimizado para la arquitectura interna de los árboles y computa valores exactos de forma sumamente eficiente. Si el modelo subyacente carece de esta estructura (como es el caso de las Máquinas de Vectores de Soporte), el sistema aplica una heurística de respaldo, decayendo de forma segura hacia un aproximador agnóstico al modelo (`Explainer`), garantizando así la robustez y continuidad del flujo de evaluación de MLOps.

Interpretación Espacial de Grafos

La explicabilidad en las Redes Neuronales sobre Grafos presenta un desafío semántico sustancialmente mayor. Mientras que en los modelos tabulares se evalúan columnas estadísticas aisladas, en las GNN resulta imperativo comprender qué subgrafo (es decir, qué transferencias específicas entre qué entidades) desencadenó la alarma algorítmica.

Para dotar al sistema de esta interpretabilidad espacial, se ha integrado el algoritmo `GNNExplainer` [37] a través de la librería PyTorch Geometric. Dada la tipología del problema, donde el objetivo es clasificar transacciones individuales (aristas) más que los nodos en sí mismos, la arquitectura hace uso de la clase `EdgeClassificationWrapper`, encargada de encapsular el codificador y el clasificador. El explicador aprende máscaras de importancia (*soft masks*) que identifican qué vecinos topológicos fueron determinantes en la predicción.

En el listado 3.5 se presenta un fragmento de la inicialización de este mecanismo en la clase `GnnExplainerModel`, demostrando la parametrización específica requerida para operar a nivel de arista (`task_level="edge"`):

```

1 self.explainer = Explainer(
2     model=self.wrapped_model,
3     algorithm=GNNExplainer(epochs=200),
4     explanation_type="model",
5     node_mask_type="attributes",
6     edge_mask_type="object",
7     model_config=dict(
8         mode="binary_classification",
9         task_level="edge",
10        return_type="raw",
11    ),
12 )

```

Listado 3.5: Fragmento de la clase `GnnExplainerModel`

Visualización y Análisis Forense

La utilidad última de la explicabilidad radica en su capacidad para ser consumida y analizada por investigadores o peritos forenses. Con este propósito, el sistema delega la representación algorítmica a la capa de presentación.

Para las arquitecturas tradicionales, la clase `TraditionalVisualizer` automatiza la generación de diagramas de resumen y de impacto (*summary and bar plots*) utilizando `matplotlib` [38]. Estos artefactos visuales permiten identificar rápidamente la direccionalidad y magnitud de las variables sintéticas precalculadas en el espacio tabular.

En contraste, la clase `GnnVisualizer` aborda la representación micro-topológica. Cuando el explicador gráfico devuelve las máscaras de atención, el visualizador aísla las K aristas con mayor coeficiente de importancia y reconstruye un subgrafo analítico haciendo uso de la librería `NetworkX` [39]. Esta proyección pondera el grosor de cada conexión de forma proporcional a su peso en la decisión del modelo, entregando a los analistas una red relacional explícita y fácilmente auditable de la cadena delictiva detectada.

3.3.5. Orquestación, Trazabilidad y Exportación

Para garantizar la reproducibilidad científica y gestionar la complejidad inherente a la experimentación iterativa con una gran cantidad de hiperparámetros, la arquitectura del sistema incorpora un módulo transversal dedicado a las operaciones de aprendizaje automático (MLOps). Este componente se responsabiliza de registrar el ciclo de vida completo de cada modelo, desde su configuración inicial hasta su persistencia a largo plazo.

Sistema de Trazabilidad con MLflow

El núcleo de la trazabilidad recae sobre la clase `ExperimentTracker`, la cual actúa como una capa de abstracción estructurada sobre la librería `MLflow`. Esta clase centraliza el registro de ejecuciones en una base de datos local `SQLite`, acoplando sistemáticamente las métricas de evaluación (como el *F1-Score* y el Área Bajo la Curva PR) y los diccionarios de hiperparámetros utilizados en cada canalización.

Un aspecto crítico de esta implementación es su capacidad para gestionar de forma agnóstica múltiples ecosistemas de modelado. Ya sea persistiendo un modelo tabular estándar, un ensamblaje basado en árboles acelerado por *hardware* (cuML), o una compleja arquitectura GNN construida sobre Pytorch, el rastreador serializa los artefactos de forma

transparente. Para lidiar con las particularidades topológicas de las Redes Neuronales sobre Grafos, el sistema implementa la clase `PyTorchModelWrapper`, encargada de consolidar el codificador de representación (*encoder*) y el clasificador final en un único módulo serializable, asegurando así la consistencia estructural requerida por MLflow.

```

1  class PyTorchModelWrapper(torch.nn.Module):
2      """Envuelve el codificador y el clasificador en un único módulo."""
3
4      def __init__(self, encoder: torch.nn.Module, classifier: torch.nn.Module):
5          """Inicializa el paquete con los submódulos."""
6          super().__init__()
7          self.encoder = encoder
8          self.classifier = classifier
9
10     def forward(self, *args: Any, **kwargs: Any):
11         """Paso adelante para la compatibilidad con la interfaz."""
12         return None

```

Listado 3.6: Definición de la clase `PyTorchModelWrapper`

Empaquetado y Exportación a la Nube

Dado que el entrenamiento de estos modelos sobre millones de transacciones se delega frecuentemente en infraestructuras de supercomputación remotas y no interactivas (tales como las máquinas NVIDIA DGX descritas en apartados anteriores), resulta imperativo asegurar que los artefactos generados se externalicen de forma segura al finalizar la ejecución. Para solventar este reto operativo, se ha desarrollado el módulo `DataSyncManager`, el cual interactúa directamente con la API de Hugging Face Hub.

Al concluir una batería de experimentos, la rutina `upload_results_to_hub` (listado 3.7) integrada en el rastreador automatiza el proceso de exportación. El sistema comprime de forma autónoma el directorio completo de artefactos de MLflow (incluyendo tanto la base de datos relacional de seguimiento como los archivos binarios de los modelos subyacentes) en un archivo empaquetado (`tar.gz`) y lo transfiere a un repositorio remoto privado. Este mecanismo completamente desatendido elimina la necesidad de intervención manual y garantiza que el registro histórico de la investigación permanezca sincronizado y accesible para su posterior análisis forense y auditoría.

3.3.6. Interfaz de Línea de Comandos y Configuración

Para garantizar la reproducibilidad y facilitar la ejecución de los distintos experimentos (tanto para modelos tradicionales como para Redes Neuronales sobre Grafos), se ha implementado un sistema de configuración declarativo y un punto de entrada unificado. Esta aproximación arquitectónica permite desacoplar la lógica de entrenamiento de la definición de los parámetros, simplificando la orquestación y el control de versiones de cada iteración experimental.

Sistema de Configuración y Definición de Hiperparámetros

La gestión de los parámetros que rigen los experimentos se centraliza mediante archivos de configuración en formato TOML. Este formato resulta altamente legible e idóneo para

```

1 def upload_results_to_hub(self, hf_repo_id: str | None = None) -> None:
2     """Comprime y sube el registro MLflow a Hugging Face Hub."""
3     repo_id = hf_repo_id or os.getenv("HF_MODEL_REPO_ID")
4     if not repo_id:
5         self.logger.warning("HF_MODEL_REPO_ID not set. Skipping MLflow upload.")
6         return
7
8     archive_name = f"mlflow_{self.experiment_name}.tar.gz"
9     archive_path = PROJECT_ROOT / "outputs" / archive_name
10
11     self.logger.info("Compressing MLflow store for upload...")
12     self._compress_mlflow_store(archive_path)
13
14     sync_manager = DataSyncManager()
15     sync_manager.upload_artifact(
16         local_path=str(archive_path),
17         repo_id=repo_id,
18         remote_filename=f"mlflow/{archive_name}",
19     )
20     self.logger.info("MLflow results uploaded to %s", repo_id)

```

Listado 3.7: Definición del método `upload_results_to_hub`

estructurar configuraciones jerárquicas. En cada archivo TOML se definen apartados clave como la selección y prefijo del conjunto de datos (`[dataset]`), la estrategia de particionado (`[split]`), y las arquitecturas a entrenar junto con sus hiperparámetros específicos (`[models.NOMBRE_MODELO]`).

Para validar e interpretar esta configuración dentro de la aplicación, se ha implementado un envoltorio fuertemente tipado utilizando `dataclasses` de Python y la librería estándar `tomllib`. Esto asegura que la configuración sea inyectada de forma segura en los distintos *pipelines*, detectando posibles errores estructurales antes del inicio de los cálculos.

A continuación, se muestra a modo ilustrativo el archivo TOML empleado para configurar uno de los experimentos con Redes Neuronales sobre Grafos:

```

1 [dataset]
2 handle = "ealtman2019/ibm-transactions-for-anti-money-laundering-aml"
3 prefix = "HI-Small"
4
5 [split]
6 test_size = 0.4
7 random_state = 42
8
9 [models.MEGA_PNA]
10 hidden_channels = 64
11 num_layers = 2
12 learning_rate = 0.001
13 epochs = 80
14 batch_size = 2048
15 loss_type = "weighted"
16 pos_weight = 5.0
17 dropout = 0.3
18 final_dropout = 0.3
19 num_neighbors = [10, 5]

```

Listado 3.8: Ejemplo de fichero de configuración de experimentos

Punto de Entrada Unificado

Para interactuar con el sistema de forma estandarizada sin necesidad de modificar el código fuente, se ha desarrollado una Interfaz de Línea de Comandos (CLI) robusta utilizando el módulo `argparse`. Este punto de entrada central expone múltiples subcomandos que dirigen el flujo de ejecución hacia el *pipeline* de procesamiento correspondiente:

- **traditional**: despliega el entrenamiento y evaluación comparativa de los modelos de *Machine Learning* clásico.
- **gnn**: inicia el ciclo de entrenamiento y validación de las arquitecturas de aprendizaje profundo sobre grafos.
- **gnn-grid**: ejecuta un proceso automatizado de búsqueda de hiperparámetros (*Grid Search*) específico para optimizar los modelos GNN.
- **fetch-results**: descarga y exporta las métricas y registros de los experimentos almacenados remotamente en un formato procesable (CSV).

El CLI requiere obligatoriamente que se especifique la ruta al archivo de configuración a utilizar mediante el argumento `--config`, integrando así ambos módulos. En el siguiente fragmento de código se detalla la inicialización y el enrutamiento de dichos subcomandos en el punto de entrada principal:

```

1 def main() -> None:
2     """Analiza los argumentos y los envía al proceso correspondiente."""
3     ProjectLogger.initialize()
4
5     parser = argparse.ArgumentParser(
6         prog="fraud-detect",
7         description="Fraud Detection: ML vs GNN comparison toolkit.",
8     )
9     subparsers = parser.add_subparsers(dest="pipeline", required=True)
10
11     # Registro de subcomandos disponibles
12     _build_traditional_parser(subparsers)
13     _build_gnn_parser(subparsers)
14     _build_gnn_grid_parser(subparsers)
15     _build_fetch_results_parser(subparsers)
16
17     args = parser.parse_args()
18
19     # Enrutamiento dinámico al pipeline correspondiente
20     dispatch = {
21         "traditional": _run_traditional,
22         "gnn": _run_gnn,
23         "gnn-grid": _run_gnn_grid,
24         "fetch-results": _run_fetch_results,
25     }
26
27     handler = dispatch.get(args.pipeline)
28     if handler is None:
29         parser.print_help()
30         sys.exit(1)
31
32     handler(args)

```

Listado 3.9: Definición del punto de entrada del programa

En conjunto, esta arquitectura minimiza la posibilidad de errores manuales en la introducción de parámetros y permite la automatización fluida e iterativa de un gran volumen de pruebas.

3.4. Desarrollo de *baselines* de *Machine Learning*

Una vez finalizado el desarrollo del sistema principal basado en Redes Neuronales sobre Grafos, se identificó la necesidad de validar empíricamente la hipótesis fundamental de este trabajo: *en escenarios complejos de fraude financiero, la topología de red y las relaciones entre entidades aportan una información crucial que los modelos tabulares clásicos no pueden capturar*.

Para demostrar esta premisa, se desarrolló un experimento adicional. El objetivo de este experimento fue establecer un rendimiento base (*baseline*) utilizando algoritmos de *Machine Learning* tradicional sobre un conjunto de datos relacional de gran escala. Para ello, se escogió el *dataset* DGraph-Fin [40], un conjunto de datos financiero extenso diseñado específicamente para la detección de anomalías en grafos.

3.4.1. Adquisición y Preprocesamiento de Datos

El primer paso del experimento consistió en la descarga y el acondicionamiento del *dataset*. Dado que los modelos tradicionales no pueden ingerir la estructura de un grafo de forma nativa, se procedió a aplanar el problema de detección de fraude convirtiéndolo en una tarea de clasificación tabular estándar.

Para lograr esto, se extrajeron únicamente las características internas de los nodos (atributos de los usuarios) y se descartó deliberadamente la matriz de adyacencia o `edge_index` (las transacciones y conexiones entre ellos). Además, se aplicaron las máscaras oficiales de entrenamiento, validación y prueba proporcionadas por el conjunto de datos para garantizar la reproducibilidad y la correcta evaluación del sistema.

```

1  # Carga del dataset DGraph-Fin
2  dataset = DGraphFin(root=root)
3  data = dataset[0]
4
5  # Extracción exclusiva de características de nodos y etiquetas mediante máscaras
6  X_train = data.x[data.train_mask].numpy()
7  y_train = data.y[data.train_mask].numpy()
8
9  X_val = data.x[data.val_mask].numpy()
10 y_val = data.y[data.val_mask].numpy()
11
12 X_test = data.x[data.test_mask].numpy()
13 y_test = data.y[data.test_mask].numpy()

```

Listado 3.10: Adquisición y preprocesamiento de datos de DGraph

3.4.2. Configuración y Entrenamiento de los Modelos

Para representar el estado del arte en clasificación tabular, se seleccionaron dos de los algoritmos basados en árboles de decisión más robustos y utilizados en la industria: *Random Forest* y *XGBoost* (*eXtreme Gradient Boosting*).

El desarrollo de ambos modelos se diseñó prestando especial atención al grave desbalanceo de clases intrínseco al dominio de la detección de fraude (donde las instancias fraudulentas representan una minoría ínfima frente a las normales).

En el caso de *Random Forest*, se utilizó el parámetro `class_weight="balanced"` para ajustar automáticamente los pesos de las clases en función de sus frecuencias. Para *XGBoost*, se calculó manualmente la proporción de pesos positivos (`scale_pos_weight`) dividiendo el número de muestras negativas entre las positivas del conjunto de entrenamiento.

```
1  # Cálculo del peso para la clase minoritaria (fraude)
2  num_neg = np.sum(y_train == 0)
3  num_pos = np.sum(y_train == 1)
4  scale_pos_weight = num_neg / num_pos
5
6  # Configuración y entrenamiento de XGBoost
7  xgb_model = XGBClassifier(
8      n_estimators=100,
9      max_depth=6,
10     learning_rate=0.1,
11     scale_pos_weight=scale_pos_weight,
12     eval_metric="aucpr",
13     random_state=42,
14 )
15
16 xgb_model.fit(X_train, y_train)
```

Listado 3.11: Manejo del desbalanceo de clases de DGraph

Ambos modelos fueron evaluados utilizando una función de métricas personalizada que calcula la exactitud (*accuracy*), precisión, *recall*, la puntuación F1 y el área bajo la curva ROC (AUC ROC). La elección de estas métricas, en especial el contraste entre el AUC y el *F1-Score*, se diseñó específicamente para evidenciar las limitaciones de evaluar a los usuarios de manera aislada, sentando así las bases para la posterior comparación con los resultados obtenidos por la arquitectura GNN.

Capítulo 4

Resultados

4.1. Características del Conjunto de Datos IBM AML

Previamente al análisis comparativo de los modelos predictivos, resulta imperativo detallar las características volumétricas y estadísticas del conjunto de datos *IBM Transactions for Anti Money Laundering (AML)*. Este simulador de transacciones bancarias sintéticas se distribuye en múltiples variantes escalonadas, diseñadas para someter a prueba la capacidad de procesamiento de las arquitecturas de detección de fraude en diferentes escenarios de estrés computacional.

Las variantes se dividen en tres magnitudes principales (*Small*, *Medium* y *Large*) y dos niveles de prevalencia de la clase minoritaria: un ratio alto de operaciones ilícitas (*High Illicit Ratio* o *HI*) y un ratio bajo (*Low Illicit Ratio* o *LI*). En la tabla 4.1 se expone un resumen comparativo de las dimensiones topológicas y tabulares de cada una de estas versiones.

Métrica	SMALL		MEDIUM		LARGE	
	HI	LI	HI	LI	HI	LI
Rango de Fechas (2022)	Sep 1-10	Sep 1-10	Sep 1-16	Sep 1-16	Ago 1 - Nov 5	Ago 1 - Nov 5
Días Abarcados	10	10	16	16	97	97
Cuentas Bancarias (Nodos)	515 K	705 K	2.07 M	2.02 M	2.11 M	2.06 M
Transacciones (Aristas)	5 M	7 M	32 M	31 M	180 M	176 M
Transacciones Ilícitas	5.1 K	4.0 K	35 K	16 K	223 K	100 K
Tasa de Fraude (1 de cada N)	981	1942	905	1948	807	1750

Tabla 4.1: Comparativa de las variantes del conjunto de datos IBM AML

Como se puede observar, el volumen de transacciones abarca desde los cinco millones en la variante *Small*, hasta un ecosistema masivo de ciento ochenta millones de aristas en la versión *Large*. Dada la naturaleza iterativa de esta investigación, la cual requiere la ejecución de procesos exhaustivos de búsqueda de hiperparámetros (*grid search*) y la validación para múltiples modelos tabulares y arquitecturas sobre grafos, la elección de la escala de datos constituye un factor crítico.

Para el desarrollo de los experimentos expuestos en los siguientes apartados, se ha seleccionado la variante HI-Small. Esta resolución se fundamenta en un compromiso pragmático

entre la representatividad estadística y las restricciones de recursos computacionales, apoyado principalmente en las siguientes consideraciones:

- **Viabilidad temporal e iterativa:** el entrenamiento de modelos topológicos profundos (como MEGA-PNA) conlleva una complejidad temporal y de memoria significativamente superior a la de los ensamblajes basados en árboles. Cargar en la memoria de vídeo (VRAM) de la GPU las matrices dispersas de adyacencia correspondientes a decenas o cientos de millones de transacciones (como exigen las versiones *Medium* o *Large*) dilataría los ciclos de entrenamiento durante semanas, limitando severamente la capacidad de experimentación ágil y la sintonización fina de la arquitectura.
- **Representatividad del desbalanceo:** pese a ser la variante más reducida, HI-Small representa un escenario de gran escala y altamente realista, abarcando diez días de operativa bancaria con más de medio millón de entidades financieras y cinco millones de interacciones dinámicas. Además, conserva una tasa de blanqueo de capitales de aproximadamente 1 caso por cada 981 transacciones (en torno al 0.1 %), lo que garantiza un entorno de desbalanceo extremo e idóneo para validar la robustez de las métricas PR-AUC y *F1-Score* utilizadas.

Por consiguiente, la variante HI-Small proporciona un desafío estructural lo suficientemente complejo para contrastar el rendimiento entre paradigmas analíticos, manteniendo los tiempos de cómputo y los requerimientos de *hardware* dentro de márgenes operativos manejables para la consecución de este proyecto.

4.2. Análisis de Modelos de *Machine Learning*

En esta sección se presentan y analizan los resultados empíricos obtenidos al evaluar los algoritmos de *Machine Learning* tradicional sobre el conjunto de datos IBM AML (variante HI-Small). Estos modelos, que actúan como línea base (*baseline*), operan exclusivamente sobre las características tabulares extraídas durante la fase de preprocesamiento, careciendo de información relacional o topológica explícita.

Como se justificó en el marco metodológico, dada la extrema asimetría entre las clases legítima y fraudulenta, la evaluación se aparta de la métrica de exactitud (*accuracy*). Predecir sistemáticamente la clase mayoritaria arroja valores de exactitud superiores al 99 %, ocultando el rendimiento real del modelo. Por consiguiente, el análisis se centra en métricas robustas frente al desbalanceo masivo: la precisión, la sensibilidad (*recall*), y la puntuación F1 (*F1-Score*).

En la tabla 4.2 se detallan las métricas alcanzadas por los clasificadores en los subconjuntos de validación y prueba.

A partir de los resultados expuestos, se extraen las siguientes conclusiones analíticas respecto al comportamiento de cada arquitectura:

- **Colapso predictivo en modelos lineales (SVM):** todos los modelos alcanzan una exactitud general superior al 99.89 %. Sin embargo, el comportamiento de la Máquina de Vectores de Soporte (SVM) ejemplifica el riesgo inherente del desbalanceo de clases. Al observar un *F1-Score* de 0.0000 tanto en validación como en prueba, se constata que el modelo ha sufrido un colapso predictivo total, convergiendo hacia una predicción constante de la clase mayoritaria («no fraude»). La incapacidad del

Modelo	Validación			Prueba		
	Precisión	Recall	F1-Score	Precisión	Recall	F1-Score
XGBoost	0.9070	0.2615	0.4059	0.8875	0.2546	0.3957
Random Forest	0.9194	0.2404	0.3812	0.9223	0.2518	0.3957
LightGBM	0.4248	0.1676	0.2404	0.4152	0.1716	0.2428
SVM	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Tabla 4.2: Resultados de los modelos tradicionales sobre el conjunto IBM AML

algoritmo para trazar un hiperplano que aisle de forma no lineal la ínfima minoría fraudulenta en el espacio tabular provoca una ceguera completa ante las anomalías.

- **Robustez de los ensamblajes basados en árboles:** los algoritmos XGBoost y *Random Forest* demostraron ser los más solventes en este entorno hostil, logrando empatar con un *F1-Score* de 0.3957 en el conjunto de prueba. *Random Forest* prioriza la minimización de falsos positivos, alcanzando una precisión sobresaliente del 92.23 %. Por su parte, XGBoost exhibe un comportamiento levemente más equilibrado, liderando marginalmente el índice de sensibilidad.

No obstante, el principal cuello de botella de ambas arquitecturas reside precisamente en su *recall* (en torno al 25 %). Esto significa que, si bien estos algoritmos son altamente precisos cuando emiten una alerta, en la práctica omiten aproximadamente el 75 % de las transacciones fraudulentas reales, lo cual resulta inaceptable para un entorno bancario de producción.

- **Bajo desempeño y ruido en LightGBM:** a pesar de representar el estado del arte en implementaciones de *Gradient Boosting* para datos tabulares, LightGBM presenta un rendimiento notablemente inferior en este conjunto, con un *F1-Score* de 0.2428 en prueba. La caída abrupta en su precisión (41.52 %) sugiere una mayor propensión a generar falsos positivos al intentar modelar heurísticas complejas, viéndose penalizado por la falta de un contexto de red estructurado.

En síntesis, el bajo índice de sensibilidad (*recall*) generalizado en los modelos tabulares corrobora la premisa teórica de esta investigación. Al carecer de la topología del grafo, los ensamblajes de árboles evalúan eventos de forma aislada y no logran propagar el riesgo a través de cadenas de transacciones coordinadas, características del blanqueo de capitales. Estas limitaciones estructurales marcan el techo de cristal del *Machine Learning* tradicional, justificando la imperiosa necesidad de evaluar el paradigma de las Redes Neuronales sobre Grafos abordado en las siguientes secciones.

4.3. Análisis de Resultados del Modelo de GNN

En esta sección se presentan y analizan los resultados empíricos obtenidos tras la ejecución de la búsqueda de hiperparámetros (*grid search*) para la arquitectura de aprendizaje profundo sobre grafos MEGA-PNA. El objetivo de esta fase de experimentación es determinar la configuración óptima que maximice la capacidad del modelo para identificar patrones fraudulentos en la red transaccional del conjunto de datos IBM AML.

Para llevar a cabo esta sintonización, se definió un espacio de búsqueda centrado en dos hiperparámetros críticos que influyen directamente en la topología de la red y en el tratamiento del desbalanceo de clases:

- **num_neighbors:** define el tamaño de la vecindad muestreada en las dos capas de convolución. Se evaluaron tres configuraciones: muestreo reducido [10, 5], intermedio [20, 10] y extenso [30, 15].
- **pos_weight:** ponderación asignada a la clase minoritaria (fraude) en la función de pérdida de entropía cruzada binaria (BCE). Se evaluaron los valores 3.0, 5.0, 7.0, 10.0 y 12.0.

En la tabla 4.3 se detallan las métricas alcanzadas en el conjunto de prueba para las quince combinaciones evaluadas, ordenadas de forma descendente en función de su puntuación F1 (*F1-Score*).

pos_weight	num_neighbors	F1-Score	PR-AUC	ROC-AUC
5,0	[10, 5]	0.2768	0.2333	0.9789
3,0	[10, 5]	0.2534	0.2125	0.9766
12,0	[10, 5]	0.2441	0.1986	0.9783
7,0	[10, 5]	0.2314	0.1888	0.9774
10,0	[10, 5]	0.2294	0.1801	0.9747
5,0	[20, 10]	0.1980	0.1319	0.9303
7,0	[20, 10]	0.1863	0.1074	0.8794
10,0	[20, 10]	0.1737	0.1262	0.9168
3,0	[20, 10]	0.1623	0.1099	0.9085
12,0	[20, 10]	0.1515	0.1179	0.9322
10,0	[30, 15]	0.1211	0.0619	0.7590
12,0	[30, 15]	0.1188	0.0575	0.7916
5,0	[30, 15]	0.1163	0.0683	0.8100
3,0	[30, 15]	0.1135	0.0570	0.7864
7,0	[30, 15]	0.0965	0.0403	0.7548

Tabla 4.3: Resultados del modelo GNN con distintos hiperparámetros

A partir de los resultados expuestos, se extraen las siguientes conclusiones analíticas respecto al comportamiento de la arquitectura GNN:

- **Degradación por sobre-suavizado (*oversmoothing*):** el hallazgo más contundente de la experimentación es el impacto negativo que produce el incremento en el número de vecinos muestreados. Como se observa en la tabla, todas las configuraciones con muestreo reducido superan sistemáticamente a las de muestreo intermedio y extenso. Al agregar información de vecindarios demasiado amplios, el modelo sufre de *oversmoothing*, un fenómeno común en las GNN donde las representaciones vectoriales de los nodos fraudulentos se diluyen y se vuelven indistinguibles de la inmensa mayoría de transacciones lícitas que los rodean.
- **Equilibrio en la ponderación de la pérdida:** la calibración del parámetro **pos_weight** revela que los valores moderados (3.0 y 5.0) logran el mejor balance entre precisión y *recall*. Un valor de 5.0 obtiene el máximo global (*F1-Score* de 0.2768),

mientras que forzar una penalización mayor (como 10.0 o 12.0) no se traduce en mejoras significativas e, incluso, empeora el rendimiento general al propiciar que el modelo sea excesivamente pesimista, disparando la tasa de falsos positivos.

- **Disparidad entre el AUC ROC y el $F1$ -Score:** es destacable que el área bajo la curva ROC (ROC-AUC) alcance valores extraordinariamente altos (rozando el 0.98 en las mejores ejecuciones). Esto indica que el modelo es muy capaz de separar globalmente ambas distribuciones. Sin embargo, la drástica caída en el área bajo la curva PR (PR-AUC) y en el $F1$ -Score demuestra empíricamente lo apuntado en los capítulos teóricos: ante un desbalanceo extremo, la métrica ROC-AUC resulta engañosa y el verdadero desafío algorítmico reside en acotar los falsos positivos sin perder sensibilidad.

4.4. Comparativa entre GNN y *Machine Learning*

Para culminar la fase de experimentación, se procede a confrontar el rendimiento de la mejor configuración obtenida para la arquitectura de grafo (MEGA-PNA) frente a los algoritmos tabulares que conforman la línea base (*baseline*). Esta comparativa resulta fundamental para evaluar si la extracción de patrones topológicos complejos compensa la sobrecarga computacional inherente al modelado de grafos en el escenario planteado por el conjunto de datos IBM AML (variante HI-Small).

Para garantizar el rigor empírico y la equidad de la comparativa, todos los modelos fueron evaluados sobre el mismo subconjunto de prueba, respetando estrictamente la segregación cronológica. En la tabla 4.4 se consolidan las métricas definitivas de rendimiento.

Modelo	Precisión	Recall	F1-Score
XGBoost	0.8875	0.2546	0.3957
Random Forest	0.9223	0.2518	0.3957
MEGA-PNA	0.3819	0.2170	0.2767
LightGBM	0.4152	0.1716	0.2428
SVM	0.0000	0.0000	0.0000

Tabla 4.4: Comparativa de rendimiento sobre IBM AML

A la luz de los datos expuestos, se extraen las siguientes conclusiones analíticas acerca del comportamiento relativo de ambas familias de algoritmos:

- **Superioridad empírica de los ensamblajes basados en árboles:** contrario a la hipótesis inicial, los modelos de *Machine Learning* tradicional (XGBoost y *Random Forest*) demostraron un rendimiento global superior al de la red neuronal sobre grafos en este ecosistema. Ambos algoritmos tabulares alcanzaron un $F1$ -Score de 0.3957, superando el 0.2767 obtenido por MEGA-PNA. Esta ventaja se cimenta principalmente en una precisión sobresaliente (en torno al 90 % frente al 38.19 % de la GNN), lo que denota una contención mucho más estricta de la tasa de falsos positivos.
- **Naturaleza sintética y densidad topológica de la variante HI-Small:** el hecho de que modelos que evalúan las transacciones de forma aislada superen a

una GNN puede explicarse por la propia configuración del simulador de IBM y las características de esta variante específica. Al tratarse de un conjunto sintético, el simulador inyecta patrones de fraude cuyas señales quedan fuertemente codificadas en los atributos tabulares de la transacción (importes, horarios, frecuencias). Los árboles de decisión explotan la no linealidad de estas variables a la perfección. Además, al ser una red de tamaño *small* (generada de forma independiente, no como un subconjunto), carece de la extrema profundidad topológica presente en versiones masivas. En este escenario menos interconectado, el mecanismo de *message passing* de la GNN no logra portar una ventaja competitiva decisiva y, por el contrario, la agregación de vecindarios mayoritariamente lícitos termina introduciendo ruido que diluye la señal de fraude.

- **Competitividad frente a estimadores más débiles:** pese a no batir a los algoritmos de *Gradient Boosting* más consolidados, la arquitectura MEGA-PNA demostró ser competente al superar con margen a modelos como LightGBM (0.2428) y SVM (0.0000). Esto ratifica que la red es capaz de procesar y aprovechar la señal relacional para identificar el fraude, aún viéndose penalizada por la abundancia de falsas alarmas (lo que lastra significativamente su *F1-Score*).
- **Compromiso operativo y coste computacional:** desde una perspectiva de arquitectura de *software* y *MLOps*, el despliegue de un sistema de inferencia basado en grafos exige una infraestructura sustancialmente mayor (ingesta en Neo4j, extracción de subgrafos, aceleración GPU con gran VRAM) en comparación con un clasificador tabular clásico. Dado que XGBoost logra identificar una proporción mayor de fraude real (*recall* del 25.46 % frente al 21.70 %) incurriendo en un número de falsos positivos abrumadoramente menor, en el escenario específico acotado por esta escala de datos, la solución basada en árboles resulta la opción más pragmática y eficiente para un entorno de producción.

4.5. Validación de la Hipótesis en DGraph-Fin

Para contrastar los hallazgos obtenidos sobre el simulador IBM AML (donde los ensamblajes basados en árboles superaron a las Redes Neuronales sobre Grafos debido a la fuerte codificación de patrones de fraude en atributos tabulares aislados), se procedió a realizar un experimento empírico adicional sobre el conjunto de datos DGraph-Fin. Al tratarse de una red financiera real a gran escala, este ecosistema presenta un desafío topológico auténtico, idóneo para validar la hipótesis fundamental de esta investigación.

En primer lugar, se evaluó el desempeño de los nodos clásicos del estado del arte (*Random Forest* y XGBoost) utilizando de forma estricta las características tabulares internas de los nodos y omitiendo deliberadamente la topología de la red (las aristas). Como se puede observar en la tabla 4.5, estos algoritmos alcanzan un Área Bajo la Curva ROC (AUC-ROC) moderado, rondando el 0.72, y una Precisión Media (*Average Precision*, AP) de aproximadamente 0.027 en el conjunto de prueba.

No obstante, esta aparente estabilidad oculta un colapso en la métrica *F1-Score* (en torno a 0.04). Al evaluar las transacciones de manera aislada, la extrema minoría de la clase fraudulenta fuerza a los modelos tabulares a generar un volumen inaceptable de falsos positivos en su intento por mantener la sensibilidad (*recall*).

Modelo	Arquitectura	AUC	AP
Random Forest	Machine Learning Tradicional	0.7209	0.0268
XGBoost	Machine Learning Tradicional	0.7233	0.0272
MLPs	Aprendizaje Profundo Tabular	0.7230	0.0270
GCN	Graph Neural Network	0.7510	0.0370
GraphSAGE	Graph Neural Network	0.7780	0.0430
TGAT	Graph Neural Network	0.7920	0.0440

Tabla 4.5: Comparativa de rendimiento sobre DGraph-Fin

Al contrastar nuestras líneas base con los resultados documentados en el artículo de referencia de DGraph-Fin [40], se evidencian diferencias sustanciales. Nuestros modelos tabulares se comportan de manera casi idéntica a los Perceptrones Multicapa (MLPs) evaluados en dicho estudio (AUC de 0.723 y AP de 0.027), lo cual demuestra que ignorar la estructura del grafo impone un techo de rendimiento infranqueable.

Por el contrario, los modelos basados en Redes Neuronales sobre Grafos que sí procesan la información topológica logran superar ampliamente esta barrera. Arquitecturas como SAGE o *Temporal Graph Attention Network* (TGAT) alcanzan un AUC superior a 0.77, llegando en el caso de TGAT a un AUC de 0.792 y un AP de 0.044. Este último valor representa un incremento relativo cercano al 63 % en la precisión media (*Average Precision*) frente a los modelos tabulares.

Confirmación de la Hipótesis Relacional

A diferencia del comportamiento observado en la sección 4.4 sobre datos sintéticos, los resultados empíricos obtenidos sobre DGraph-Fin confirman de manera categórica la tesis central de este trabajo: *el fraude financiero en ecosistemas reales es un fenómeno intrínsecamente relacional*.

Una entidad o usuario fraudulento puede presentar características individuales que aparenten total normalidad a los ojos de un algoritmo XGBoost o *Random Forest*, evadiendo fácilmente la clasificación. Sin embargo, su exposición estructural y sus conexiones directas e indirectas con clústeres de actividad anómala conocida son los delatores que permiten a las GNN filtrar de manera efectiva los falsos positivos.

En definitiva, esta comparación corrobora que, ante tipologías complejas de fraude como el blanqueo de capitales coordinado, la agregación de vecindarios y el paso de mensajes (*message passing*) que ofrecen las GNN superan las limitaciones de la evaluación aislada característica del *Machine Learning* tradicional.

4.6. Explicabilidad en Modelos de *Gradient Boosting*

Para comprender en profundidad el comportamiento de los modelos tabulares evaluados y auditar la lógica subyacente a sus predicciones, se ha recurrido a técnicas de Inteligencia Artificial Explicable (XAI), concretamente mediante la extracción de valores SHAP (*SHapley Additive exPlanations*). Este análisis resulta fundamental para diagnosticar por qué

arquitecturas como LightGBM sufrieron una caída de rendimiento tan acusada frente a XGBoost en el conjunto de prueba, tal y como se evidenció en la sección 4.2.

A continuación, se examina la contribución de las características tabulares en las inferencias de los modelos de *Gradient Boosting*, empleando para ello diagramas de impacto global (barras) y de dispersión (resumen).

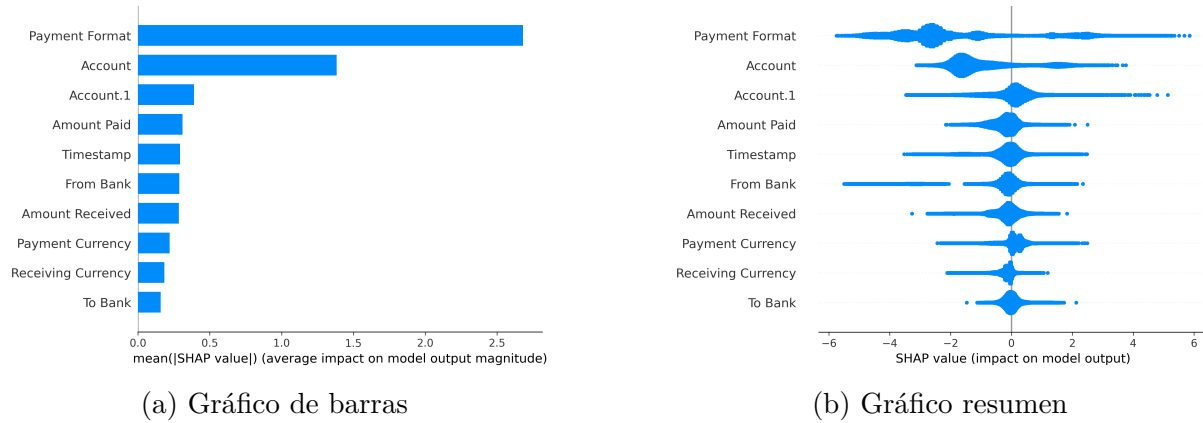


Figura 4.1: Valores SHAP de XGBoost

En primer lugar, el análisis de explicabilidad para el modelo XGBoost revela una distribución de importancia relativamente lógica dadas las limitaciones del enfoque tabular. Como se puede observar en los diagramas correspondientes, la característica **Payment Format** se erige como el atributo más determinante, seguido a cierta distancia por los identificadores de cuenta. La dispersión de los valores SHAP indica que ciertas modalidades de pago y cuentas específicas incrementan sustancialmente la probabilidad de que una transacción sea clasificada como fraudulenta. Aunque el modelo carece de contexto topológico, es capaz de trazar heurísticas útiles basadas en el canal de pago y el historial aislado de las cuentas, lo que justifica su robusto desempeño y su capacidad para mantener un *F1-Score* competitivo.

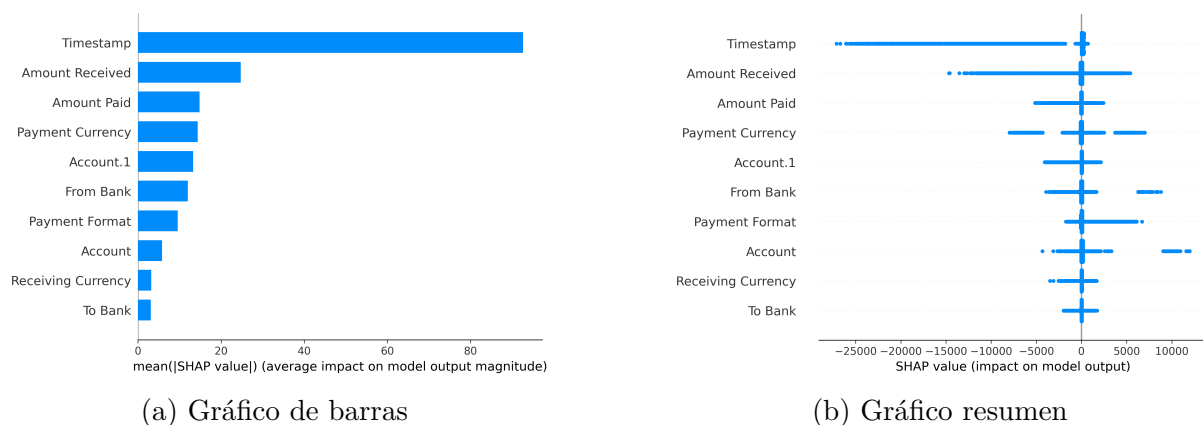


Figura 4.2: Valores SHAP de LightGBM

Por el contrario, los artefactos visuales generados para LightGBM exponen un escenario diametralmente opuesto y explican el origen de su bajo rendimiento. El diagrama de barras de LightGBM muestra una dominancia absoluta y desproporcionada de la variable

Timestamp (marca de tiempo), cuyo impacto medio en la magnitud de la salida del modelo supera en varios órdenes de magnitud al resto de las características.

Al observar el gráfico de resumen (*SHAP summary plot*) para LightGBM, se constata que la red de árboles de decisión ha generado reglas fuertemente condicionadas a instantes temporales concretos, asignando valores SHAP extremos (tanto positivos como negativos) en función del momento exacto en que se ejecutó la transacción. Este fenómeno es un síntoma inequívoco de sobreajuste (*overfitting*): el modelo no ha aprendido patrones generalizables de fraude, sino que ha memorizado las marcas de tiempo exactas en las que ocurrieron las anomalías en el conjunto de entrenamiento. Al enfrentarse a datos futuros en el conjunto de prueba, estas reglas temporales estrictas pierden toda su validez, provocando una cascada de clasificaciones erróneas y el consecuente colapso de la precisión detallado en la tabla 4.2.

En conclusión, la explicabilidad algorítmica demuestra empíricamente la fragilidad de los enfoques de *Machine Learning* tradicional en este dominio. Mientras XGBoost logra extraer el máximo valor posible del espacio tabular apoyándose en formatos de pago, arquitecturas como LightGBM sucumben ante la dimensionalidad temporal, intentando compensar la falta de visión topológica (quién transfiere a quién) con correlaciones espurias (cuándo se transfiere). Estos hallazgos reafirman la necesidad de emplear Redes Neuronales sobre Grafos, capaces de auditar la estructura real del blanqueo de capitales más allá de los atributos bidimensionales y estadísticos.

4.7. Explicabilidad en Modelos de GNN

A diferencia de los algoritmos de *Machine Learning* tradicional, donde la explicabilidad se vertebra en torno al impacto aislado de características tabulares, la interpretabilidad en las Redes Neuronales sobre Grafos exige una disección topológica. Para auditar las inferencias de la arquitectura MEGA-PNA y comprender el razonamiento subyacente a sus predicciones, se ha hecho uso del algoritmo **GNNExplainer**. Este módulo identifica qué subgrafo computacional (qué vecindario y conexiones específicas) fue determinante para que el modelo clasificara una transacción como fraudulenta.

En la figura 4.3 se presenta el artefacto visual extraído por la clase **GnnVisualizer** tras analizar una transacción anómala representativa (arista 13229) en el conjunto de prueba.

El grafo resultante proyecta las conexiones ponderando el grosor de cada arista en función de su coeficiente de importancia en la decisión final de la red. En este escenario empírico, a pesar de que el explicador rastrea el vecindario en busca de las 20 interacciones más relevantes (*top 20 edges*), el resultado aísla de manera contundente una única conexión directa y predominante entre las entidades 82757 y 268330.

Este artefacto visual forense resulta sumamente revelador y permite extraer dos conclusiones fundamentales sobre el comportamiento del modelo GNN entrenado:

- **Localización de la señal de fraude:** para esta tipología de transacciones anómalas, la arquitectura ha determinado que la propia interacción directa entre las cuentas origen y destino concentra la práctica totalidad de la carga semántica necesaria para detonar la alarma. El modelo no ha requerido propagar el riesgo a través de complejas cadenas de lavado a múltiples «saltos» de distancia (*multi-hop*), sino que

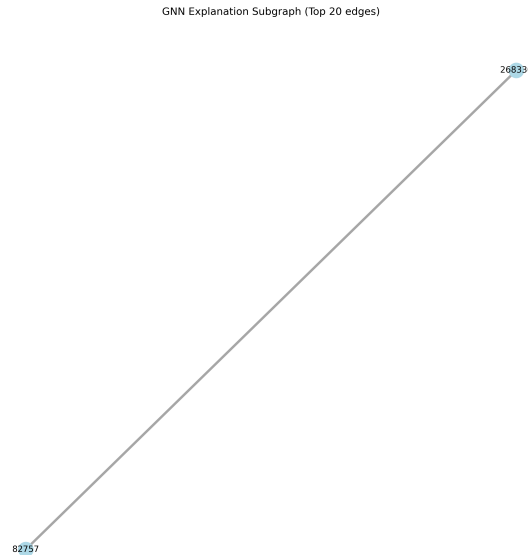


Figura 4.3: Aristas más importantes según la explicación de la GNN

los atributos embebidos en ese enlace directo resultaron suficientes para aislar la anomalía.

- **Confirmación empírica del *oversmoothing*:** esta focalización extrema en el enlace más local es estrictamente coherente con los hallazgos documentados en la fase de sintonización de hiperparámetros (sección 4.3). En dicho apartado se demostró que incrementar el tamaño de la vecindad muestreada (parámetro `num_neighbors`) producía una degradación sistemática del rendimiento debido al sobre-suavizado (*oversmoothing*). La explicabilidad visual corrobora ahora este fenómeno: dado que la señal de fraude está tan focalizada, la agregación de vecindarios extensos compuestos mayoritariamente por transacciones lícitas termina actuando como ruido, diluyendo las características distintivas de los escasos nodos fraudulentos.

En síntesis, la aplicación de Inteligencia Artificial Explicable sobre la topología de la red no solo cumple con el requisito regulatorio de aportar transparencia a la «caja negra» neuronal, sino que actúa como una herramienta de diagnóstico estructural invaluable, validando de forma visual las decisiones tomadas durante el diseño arquitectónico del sistema.

Capítulo 5

Conclusiones

La presente investigación ha abordado el desafío de la detección de fraude financiero, concretamente el blanqueo de capitales, mediante una evaluación comparativa exhaustiva entre los modelos de *Machine Learning* tradicional y las arquitecturas de aprendizaje profundo sobre grafos (GNN). A partir de la fundamentación teórica, el desarrollo del ecosistema de experimentación y el análisis riguroso de los resultados empíricos, se extraen las siguientes conclusiones fundamentales:

Limitaciones topológicas del *Machine Learning* tradicional

La validación empírica sobre el conjunto de datos real DGraph-Fin confirmó categóricamente la hipótesis central de este trabajo: el fraude financiero complejo es un fenómeno intrínsecamente relacional. Al evaluar las transacciones de manera aislada y carecer de visión topológica, los modelos basados en árboles de decisión (XGBoost y *Random Forest*) sufrieron un colapso predictivo en la métrica *F1-Score* (en torno a 0.04). Por el contrario, las arquitecturas gráficas superaron el techo de rendimiento tabular, demostrando que la exposición estructural y las conexiones con clústeres de actividad anómala son los verdaderos delatores que permiten filtrar los falsos positivos en entornos reales.

Impacto de la naturaleza sintética de los datos en el rendimiento de la GNN

A pesar de la superioridad teórica de los grafos, los resultados sobre la variante HI-Small del simulador sintético IBM AML revelaron una superioridad empírica de los ensamblajes tradicionales. Algoritmos como XGBoost alcanzaron un *F1-Score* de 0.3957, superando el 0.2767 obtenido por la red MEGA-PNA. Esto evidencia que, en entornos sintéticos donde los patrones de fraude se inyectan con una fuerte codificación en atributos tabulares aislados (como importes o formatos de pago), los árboles de decisión explotan la no linealidad de manera sobresaliente. En este escenario de menor interconexión topológica, el mecanismo de *message passing* de la GNN no logró aportar una ventaja decisiva.

Degradación por *oversmoothing* en Redes Neuronales sobre Grafos

Durante la sintonización de hiperparámetros de la arquitectura MEGA-PNA, se demostró empíricamente el impacto negativo del sobre-suavizado (*oversmoothing*). Se constató que incrementar el tamaño de la vecindad muestreada producía una degradación sistemática del rendimiento, ya que las representaciones vectoriales de los escasos nodos fraudulentos

se diluían al agregar información de vecindarios mayoritariamente lícitos. Esto obliga a mantener hiperparámetros de muestreo reducidos para no comprometer la sensibilidad del modelo.

El valor crítico de la Inteligencia Artificial Explicable (XAI)

La integración de módulos de explicabilidad ha resultado ser una herramienta de diagnóstico forense invaluable. En los modelos tabulares, los valores SHAP permitieron descubrir el sobreajuste (*overfitting*) severo de la arquitectura LightGBM, la cual había memorizado marcas de tiempo exactas en lugar de aprender generalizaciones. En el ámbito topológico, el uso de GNNExplainer corroboró visualmente que la señal de fraude se localizaba de forma contundente en conexiones directas, validando empíricamente el fenómeno de *oversmoothing* al demostrar que no era necesario propagar el riesgo a múltiples «saltos» de distancia para identificar la anomalía.

Complejidad operativa y el compromiso del entorno de producción

Desde la perspectiva de la ingeniería de software, el despliegue de sistemas de inferencia basados en grafos exige una infraestructura sustancialmente mayor a la del *Machine Learning* tradicional. La necesidad de orquestar bases de datos de grafos como Neo4j, extraer subgrafos en tiempo real y requerir aceleración GPU con alta memoria de vídeo (VRAM), contrasta con la eficiencia pragmática de clasificadores como XGBoost. En escenarios donde la densidad relacional es baja, la solución basada en árboles sigue representando la opción más eficiente en términos de coste computacional para un entorno de producción.

5.1. Líneas de trabajo futuro

A la luz de las conclusiones obtenidas y de las limitaciones identificadas durante el desarrollo de este trabajo, se proponen las siguientes líneas de investigación para dar continuidad al estudio de la detección de fraude mediante inteligencia artificial:

- **Evaluación sobre escalas topológicas masivas:** dado que la variante HI-Small del conjunto IBM AML no presentó la profundidad relacional suficiente para beneficiar a las GNN, se propone escalar los experimentos hacia las variantes Medium y Large. El procesamiento de redes con hasta ciento ochenta millones de aristas permitiría auditar si ecosistemas de tal magnitud revelan cadenas de lavado de dinero multicapa que resulten totalmente invisibles para los modelos tabulares.
- **Desarrollo de arquitecturas híbridas:** con el objetivo de aunar las fortalezas de ambos paradigmas, una línea prometedora consistiría en extraer los *embeddings* latentes generados por las capas convolucionales de MEGA-PNA e inyectarlos como características sintéticas en algoritmos de *Gradient Boosting* como XGBoost. Esto permitiría aprovechar la inigualable contención de falsos positivos de los árboles de decisión, enriqueciéndolos con el contexto de red.
- **Implementación de Redes Neuronales sobre Grafos Dinámicos (DGNN):** el fraude financiero ocurre de manera secuencial y evoluciona rápidamente. Se plantea la adopción de arquitecturas que incorporen componentes espacio-temporales,

actualizando los pesos de las aristas en tiempo real sin necesidad de reentrenar la topología completa.

- **Generación de explicaciones en lenguaje natural:** aunque herramientas como SHAP y GNNExplainer dotan de transparencia matemática al sistema, sus salidas siguen requiriendo interpretación técnica. Una vía de trabajo futuro sería acoplar Modelos Fundacionales de Lenguajes (LLM) que traduzcan los subgrafos de atención y las contribuciones de características en informes de auditoría forense redactados en lenguaje natural para los analistas de cumplimiento bancario.

Capítulo 6

Summary

This research has addressed the challenge of financial fraud detection, specifically money laundering, through a comprehensive comparative evaluation between traditional Machine Learning models and Graph Neural Network architectures. Based on the theoretical foundation, the development of the experimental ecosystem, and the rigorous analysis of the empirical results, the following fundamental conclusions are drawn:

Topological limitations of traditional Machine Learning

Empirical validation on the real-world DGraph-Fin dataset categorically confirmed the central hypothesis of this work: complex financial fraud is an intrinsically relational phenomenon. By evaluating transactions in isolation and lacking topological vision, decision tree-based models (XGBoost and *Random Forest*) suffered a predictive collapse in the F1-Score metric (around 0.04). Conversely, graph architectures overcame the tabular performance ceiling, demonstrating that structural exposure and connections to clusters of anomalous activity are the true telltales that allow filtering out false positives in real environments.

Impact of the synthetic nature of the data on GNN performance

Despite the theoretical superiority of graphs, the results on the HI-Small variant of the synthetic IBM AML simulator revealed an empirical superiority of traditional ensembles. Algorithms such as XGBoost achieved an F1-Score of 0.3957, outperforming the 0.2767 obtained by the MEGA-PNA network. This evidences that, in synthetic environments where fraud patterns are injected with a strong encoding in isolated tabular attributes (such as amounts or payment formats), decision trees exploit non-linearity outstandingly. In this scenario of lower topological interconnection, the GNN’s message passing mechanism failed to provide a decisive advantage.

Degradation by oversmoothing in graph neural networks

During the hyperparameter tuning of the MEGA-PNA architecture, the negative impact of oversmoothing was empirically demonstrated. It was observed that increasing the size of the sampled neighborhood produced a systematic performance degradation, as the vector representations of the scarce fraudulent nodes were diluted when aggregating information

from predominantly legitimate neighborhoods. This forces the maintenance of reduced sampling hyperparameters so as not to compromise the model's sensitivity.

The critical value of Explainable AI (XAI)

The integration of explainability modules has proven to be an invaluable forensic diagnostic tool. In tabular models, SHAP values allowed discovering the severe overfitting of the LightGBM architecture, which had memorized exact Timestamps instead of learning generalizations. In the topological domain, the use of GNNExplainer visually corroborated that the fraud signal was strongly localized in direct connections, empirically validating the oversmoothing phenomenon by demonstrating that it was not necessary to propagate the risk multiple "hops" away to identify the anomaly.

Operational complexity and the production environment compromise

From a software engineering perspective, the deployment of graph-based inference systems requires a substantially larger infrastructure than traditional Machine Learning. The need to orchestrate graph databases such as Neo4j, extract subgraphs in real time, and require GPU acceleration with high Video RAM (VRAM), contrasts with the pragmatic efficiency of classifiers like XGBoost. In scenarios where relational density is low, the tree-based solution remains the most efficient option in terms of computational cost for a production environment.

6.1. Future Work

In light of the conclusions drawn and the limitations identified during the development of this Bachelor's Thesis, the following research lines are proposed to continue the study of fraud detection using artificial intelligence:

- **Evaluation on massive topological scales:** Given that the HI-Small variant of the IBM AML dataset did not present sufficient relational depth to benefit GNNs, it is proposed to scale the experiments towards the Medium and Large variants. Processing networks with up to one hundred and eighty million edges would allow auditing whether ecosystems of such magnitude reveal multi-layer money laundering chains that remain totally invisible to tabular models.
- **Development of hybrid architectures:** Aiming to combine the strengths of both paradigms, a promising line would be to extract the latent embeddings generated by the convolutional layers of MEGA-PNA and inject them as synthetic features into Gradient Boosting algorithms such as XGBoost. This would allow leveraging the unparalleled false positive containment of decision trees, enriching them with network context.
- **Implementation of Dynamic Graph Neural Networks (DGNNs):** Financial fraud occurs sequentially and evolves rapidly. The adoption of architectures that incorporate spatio-temporal components is proposed, updating edge weights in real-time without the need to retrain the entire topology.
- **Generation of natural language explanations:** Although tools like SHAP and GNNExplainer provide mathematical transparency to the system, their outputs

still require technical interpretation. A future line of work would be to couple Large Language Models (LLMs) that translate attention subgraphs and feature contributions into forensic audit reports written in natural language for banking compliance analysts.

Capítulo 7

Presupuesto

El presente capítulo detalla la valoración económica estimada para la ejecución de este trabajo. Para la elaboración del presupuesto se han cuantificado los recursos humanos, el *hardware* empleado para el desarrollo y entrenamiento de los modelos, así como las licencias de *software* necesarias. Aunque diversos recursos, como la infraestructura de supercomputación y las herramientas de código abierto, no han supuesto un desembolso económico directo, se han valorado a precios de mercado para reflejar el coste real de un proyecto de esta envergadura en un entorno empresarial.

7.1. Costes de Personal

El desarrollo del proyecto ha abarcado las fases de investigación, preprocesamiento de datos, diseño arquitectónico de las Redes Neuronales sobre Grafos, entrenamiento, evaluación y redacción de la memoria. Se estima una carga de trabajo equivalente a 300 horas (12 créditos ECTS a 25 horas por crédito), con una tarifa estándar estimada de 30.00 €/hora.

Perfil Profesional	Horas Estimadas	Coste Unitario (€/h)	Coste Total (€)
Ingeniero Informático	300	30.00	9000.00
Subtotal Personal			9000.00

Tabla 7.1: Presupuesto de personal

7.2. Costes de *Hardware*

El proyecto ha requerido una arquitectura de *hardware* híbrida. En primer lugar, la fase de preprocesamiento, escritura de código y pruebas iniciales se ha ejecutado de manera local utilizando un equipo portátil Lenovo V15 G4 AMN. Para calcular su coste, se aplica una amortización lineal considerando un valor de adquisición de 600.00 €, una vida útil de 4 años y un periodo de uso en el proyecto de 6 meses.

En segundo lugar, debido a la carga computacional masiva que exige el entrenamiento de la arquitectura MEGA-PNA sobre grafos de millones de nodos, se ha utilizado la infraestructura de alto rendimiento NVIDIA DGX Spark cedida por la Cátedra BOB.

Para estimar este coste en el mercado libre, se utiliza como referencia el precio de alquiler de instancias en la nube equipadas con GPU de características similares durante un total estimado de 200 horas de cómputo intensivo.

Concepto	Coste Unitario	Uso Estimado	Coste Total (€)
Portátil Lenovo	600.00 (€)	6 meses	75.00
Cómputo GPU	12.00 (€/h)	200 horas	2400.00
Subtotal Hardware			2475.00

Tabla 7.2: Presupuesto de *Hardware*

7.3. Costes de *Software*

Todo el ecosistema de experimentación desarrollado en este trabajo se fundamenta en tecnologías de código abierto e infraestructuras *open source* o de uso gratuito para fines académicos.

7.4. Resumen del Presupuesto

A continuación, se consolida el total de las partidas descritas anteriormente. Para el cálculo del importe final, se aplica el Impuesto General Indirecto Canario (IGIC) vigente del 7 %, aplicable a los servicios y desarrollos realizados en la Comunidad Autónoma de Canarias.

Partida Presupuestaria	Importe (€)
Costes de Personal	9000,00
Costes de Hardware	2475,00
Costes de Software	0,00
Base Imponible	11475.00
IGIC (7 %)	803,25
Coste Total del Proyecto	12278.25

Tabla 7.3: Presupuesto total

Bibliografía

- [1] Dawei Cheng et al. «Graph Neural Networks for Financial Fraud Detection: A Review». En: *Frontiers of Computer Science* 19.9 (sep. de 2025), pág. 199609. ISSN: 2095-2228, 2095-2236. DOI: [10.1007/s11704-024-40474-y](https://doi.org/10.1007/s11704-024-40474-y). URL: <https://link.springer.com/10.1007/s11704-024-40474-y>.
- [2] Naim, Brad Rees y Summer Liu. *Supercharging Fraud Detection in Financial Services with Graph Neural Networks*. NVIDIA Technical Blog. 3 de jun. de 2025. URL: <https://developer.nvidia.com/blog/supercharging-fraud-detection-in-financial-services-with-graph-neural-networks/>.
- [3] Erik Altman et al. *Realistic Synthetic Financial Transactions for Anti-Money Laundering Models*. 2023. DOI: [10.48550/ARXIV.2306.16424](https://doi.org/10.48550/ARXIV.2306.16424). URL: <https://arxiv.org/abs/2306.16424>. Prepublicado.
- [4] Irin Sultana et al. *detectGNN: Harnessing Graph Neural Networks for Enhanced Fraud Detection in Credit Card Transactions*. 2025. DOI: [10.48550/ARXIV.2503.22681](https://doi.org/10.48550/ARXIV.2503.22681). URL: <https://arxiv.org/abs/2503.22681>. Prepublicado.
- [5] *A Graph-Based Approach to Financial Fraud Detection*. Neo4j Graph Data Platform. URL: <https://neo4j.com/developer/industry-use-cases/finserv/retail-banking/ieee-cis-fraud-graphs/>.
- [6] Iftekhar Rasul et al. «Detecting Financial Fraud in Real-Time Transactions Using Graph Neural Networks and Anomaly Detection». En: *Journal of Economics, Finance and Accounting Studies* 6.1 (25 de feb. de 2024), págs. 131-142. ISSN: 2709-0809. DOI: [10.32996/jefas.2024.6.1.13](https://doi.org/10.32996/jefas.2024.6.1.13). URL: <https://al-kindipublisher.com/index.php/jefas/article/view/10159>.
- [7] Sirui Lu. «Graph Neural Network Model in Financial Fraud Detection». En: *2025 2nd International Conference on Intelligent Computing and Robotics (ICICR)*. 2025 2nd International Conference on Intelligent Computing and Robotics (ICICR). Dalian, China: IEEE, 16 de mayo de 2025, págs. 998-1002. ISBN: 979-8-3315-4429-4. DOI: [10.1109/ICICR65456.2025.00176](https://doi.org/10.1109/ICICR65456.2025.00176). URL: <https://ieeexplore.ieee.org/document/11172714/>.
- [8] Fahad Almalki y Mehedi Masud. *Financial Fraud Detection Using Explainable AI and Stacking Ensemble Methods*. Ver. 1. 2025. DOI: [10.48550/ARXIV.2505.10050](https://doi.org/10.48550/ARXIV.2505.10050). URL: <https://arxiv.org/abs/2505.10050>. Prepublicado.
- [9] Lina Ni et al. «HMOA-GNN: Adaptive Adversarial GraphSAGE with Hierarchical Hybrid Sampling and Metric-Optimized Graph Construction for Credit Card Fraud Detection». En: *Scientific Reports* 15.1 (2 de dic. de 2025), pág. 43005. ISSN: 2045-2322. DOI: [10.1038/s41598-025-27010-z](https://doi.org/10.1038/s41598-025-27010-z). URL: <https://www.nature.com/articles/s41598-025-27010-z>.
- [10] Fawaz Khaled Alarfaj y Shabnam Shahzadi. «Enhancing Fraud Detection in Banking With Deep Learning: Graph Neural Networks and Autoencoders for Real-Time

- Credit Card Fraud Prevention». En: *IEEE Access* 13 (2025), págs. 20633-20646. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2024.3466288](https://doi.org/10.1109/ACCESS.2024.3466288). URL: <https://ieeexplore.ieee.org/document/10689393/>.
- [11] Firman Multi. *IEEE-CIS Fraud Detection*. URL: <https://kaggle.com/code/firmanmulti/ieee-cis-fraud-detection>.
- [12] N. V. Chawla et al. «SMOTE: Synthetic Minority Over-sampling Technique». En: *Journal of Artificial Intelligence Research* 16 (1 de jun. de 2002), págs. 321-357. ISSN: 1076-9757. DOI: [10.1613/jair.953](https://doi.org/10.1613/jair.953). URL: <https://www.jair.org/index.php/jair/article/view/10302>.
- [13] Haibo He et al. «ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning». En: *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*. 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence). Jun. de 2008, págs. 1322-1328. DOI: [10.1109/IJCNN.2008.4633969](https://doi.org/10.1109/IJCNN.2008.4633969). URL: <https://ieeexplore.ieee.org/document/4633969>.
- [14] Erik Altman. *IBM Transactions for Anti Money Laundering (AML)*. URL: <https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>.
- [15] Imaad Mahmood. *Fraud Guard Synthetic 2025*. URL: <https://www.kaggle.com/datasets/imaadmahmood/fraud-guard-synthetic-2025>.
- [16] Erik Altman y Steve Martinelli. *IBM/AML-Data*. International Business Machines, 23 de feb. de 2026. URL: <https://github.com/IBM/AML-Data>.
- [17] Dirk Merkel. «Docker: Lightweight Linux Containers for Consistent Development and Deployment». En: *Linux J*. 2014.239 (1 de mar. de 2014), 2:2. ISSN: 1075-3583.
- [18] *The Neo4j Graph Data Science Library Manual*. Neo4j Graph Data Platform. URL: <https://neo4j.com/docs/graph-data-science/2026.05/>.
- [19] *Cátedra Cajasieta BOB*. URL: <https://www.ull.es/catedras/catedrabob/>.
- [20] Sebastián Ramírez. *FastAPI Documentation*. URL: <https://fastapi.tiangolo.com/>.
- [21] Ritchie Vink. *Pola-Rs/Polars*. Polars, 21 de abr. de 2026. URL: <https://github.com/pola-rs/polars>.
- [22] Fabian Pedregosa et al. «Scikit-Learn: Machine Learning in Python». En: (2012). DOI: [10.48550/ARXIV.1201.0490](https://doi.org/10.48550/ARXIV.1201.0490). URL: <https://arxiv.org/abs/1201.0490>.
- [23] Tianqi Chen y Carlos Guestrin. «XGBoost: A Scalable Tree Boosting System». En: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16: The 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco California USA: ACM, 13 de ago. de 2016, págs. 785-794. ISBN: 978-1-4503-4232-2. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785). URL: <https://dl.acm.org/doi/10.1145/2939672.2939785>.
- [24] Guolin Ke et al. «LightGBM: A Highly Efficient Gradient Boosting Decision Tree». En: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.
- [25] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. DOI: [10.48550/ARXIV.1912.01703](https://doi.org/10.48550/ARXIV.1912.01703). URL: <https://arxiv.org/abs/1912.01703>. Prepublicado.

- [26] Matthias Fey y Jan Eric Lenssen. *Fast Graph Representation Learning with PyTorch Geometric*. 2019. DOI: [10.48550/ARXIV.1903.02428](https://doi.org/10.48550/ARXIV.1903.02428). URL: <https://arxiv.org/abs/1903.02428>. Prepublicado.
- [27] Scott Lundberg y Su-In Lee. *A Unified Approach to Interpreting Model Predictions*. 2017. DOI: [10.48550/ARXIV.1705.07874](https://doi.org/10.48550/ARXIV.1705.07874). URL: <https://arxiv.org/abs/1705.07874>. Prepublicado.
- [28] M. Zaharia et al. «Accelerating the Machine Learning Lifecycle with MLflow». En: *IEEE Data Eng. Bull.* (2018). URL: <https://www.semanticscholar.org/paper/Accelerating-the-Machine-Learning-Lifecycle-with-Zaharia-Chen/b2e0b79e6f180af2e0e559f2b1faba66b2bd578a>.
- [29] Luca Pouget. *Huggingface/Huggingface_hub*. Hugging Face, 21 de abr. de 2026. URL: https://github.com/huggingface/huggingface_hub.
- [30] Vincent Roseberry. *Kaggle/Kagglehub*. Kaggle, 25 de jun. de 2026. URL: <https://github.com/Kaggle/kagglehub>.
- [31] Charlie Marsh. *Ruff Documentation*. URL: <https://docs.astral.sh/ruff/>.
- [32] Bruno Oliveira. *Pytest-Dev/Pytest*. pytest-dev, 21 de abr. de 2026. URL: <https://github.com/pytest-dev/pytest>.
- [33] Dante Dessavre. *cuML's Documentation*. URL: <https://docs.rapids.ai/api/cuml/stable/>.
- [34] William L. Hamilton, Rex Ying y Jure Leskovec. *Inductive Representation Learning on Large Graphs*. 10 de sep. de 2018. DOI: [10.48550/arXiv.1706.02216](https://doi.org/10.48550/arXiv.1706.02216). arXiv: [1706.02216](https://arxiv.org/abs/1706.02216) [cs.SI]. URL: <http://arxiv.org/abs/1706.02216>. Prepublicado.
- [35] Shaked Brody, Uri Alon y Eran Yahav. *How Attentive Are Graph Attention Networks?* 31 de ene. de 2022. DOI: [10.48550/arXiv.2105.14491](https://doi.org/10.48550/arXiv.2105.14491). arXiv: [2105.14491](https://arxiv.org/abs/2105.14491) [cs.LG]. URL: <http://arxiv.org/abs/2105.14491>. Prepublicado.
- [36] Gabriele Corso et al. *Principal Neighbourhood Aggregation for Graph Nets*. 31 de dic. de 2020. DOI: [10.48550/arXiv.2004.05718](https://doi.org/10.48550/arXiv.2004.05718). arXiv: [2004.05718](https://arxiv.org/abs/2004.05718) [cs.LG]. URL: <http://arxiv.org/abs/2004.05718>. Prepublicado.
- [37] Rex Ying et al. *GNExplainer: Generating Explanations for Graph Neural Networks*. 13 de nov. de 2019. DOI: [10.48550/arXiv.1903.03894](https://doi.org/10.48550/arXiv.1903.03894). arXiv: [1903.03894](https://arxiv.org/abs/1903.03894) [cs.LG]. URL: <http://arxiv.org/abs/1903.03894>. Prepublicado.
- [38] Thomas Caswell. *Matplotlib Documentation*. URL: <https://matplotlib.org/stable/api/index>.
- [39] Aric Hagberg. *NetworkX Documentation*. URL: <https://networkx.org/en/>.
- [40] Xuanwen Huang et al. *DGraph: A Large-Scale Financial Dataset for Graph Anomaly Detection*. 9 de jun. de 2023. DOI: [10.48550/arXiv.2207.03579](https://doi.org/10.48550/arXiv.2207.03579). arXiv: [2207.03579](https://arxiv.org/abs/2207.03579) [cs.SI]. URL: <http://arxiv.org/abs/2207.03579>. Prepublicado.